

Министерство образования и науки Пермского края
краевое государственное автономное профессиональное образовательное
учреждение «Пермский авиационный техникум им. А.Д. Швецова»

ДИПЛОМНЫЙ ПРОЕКТ

Пояснительная записка

Разработка программного обеспечения «Создание и обучение моделей
распознавания речи»

АТДП.09.02.07.20.404.ПЗ

Руководитель

(подпись, дата)

О.В. Ситчихин
(И.О. Фамилия)

Консультант по
экономической части

(подпись, дата)

И.Л. Мишакина
(И.О. Фамилия)

Рецензент

(подпись, дата)

А.В. Басалаев
(И.О. Фамилия)

Студент, гр. ИСП-20-4

(подпись, дата)

А.Н. Галанов
(И.О. Фамилия)

ЗАДАНИЕ

на выполнение дипломного проекта

специальность 09.02.07 «Информационные системы и программирование»

Квалификация: Программист

Студенту группы ИСП-20-4 Галанов Александр Николаевич

шифр группы

фамилия, имя, отчество

Тема проекта: Разработка программного обеспечения «Создание и обучение моделей распознавания речи»

Требования к проекту:

Программное обеспечение должно:

- позволять создавать и обучать модели распознавания речи,
- предоставлять возможность настройки параметров модели,
- предоставлять возможность составления словарей модели,
- позволять обучать модели на собственных аудиофайлах,
- включать мониторинг и визуализацию процесса обучения модели,
- включать возможность тестирования модели,
- позволять рассчитывать точность распознавания модели,
- позволять транскрибировать аудиофайлы,
- иметь интуитивно понятный пользовательский интерфейс,
- включать документацию и руководства пользователя.

Содержание пояснительной записки

Введение

1. Анализ предметной области
2. Техническое задание
3. Моделирование предметной области
4. Реализация разработки
 - 4.1. Программные средства реализации проекта
 - 4.2. Структура информационной системы
 - 4.3. Описание программный модулей
5. Тестирование разработанного продукта
 - 5.1. Анализ и выбор методов тестирования
 - 5.2. Результаты тестирования

6. Экономическое обоснование разработки

7. Внедрение и установка информационной системы

Заключение

Список использованных источников (литературы)

Приложения: код библиотеки, руководство пользователя.

Содержание графической части

UML диаграммы, структура программы и результаты тестирования представлены в тексте пояснительной записки. Код библиотеки и интерфейс приложения представлены в приложениях.

Содержание диска

- Разработанный программный продукт
- Пояснительная записка
- Презентация для защиты

Председатель ЦМК	(подпись)	Е.Г. Аристова
Руководитель проекта	(подпись)	О.В. Ситчихин
Исполнитель проекта	(подпись)	А.Н. Галанов
Дата выдачи задания	15.04.2024 г	
Дата окончания проектирования	08.06.2024 г	

Содержание

Введение	5
1 Анализ предметной области	6
2 Техническое задание	8
3 Моделирование предметной области	13
4 Реализация разработки	17
4.1 Программные средства реализации проекта	17
4.2 Структура информационной системы	19
4.3 Описание программных модулей	26
5 Тестирование разработанного продукта	30
5.1 Анализ и выбор методов тестирования	30
5.2 Результаты тестирования	31
6 Экономическое обоснование разработки	35
7 Внедрение и установка информационной системы	43
Заключение	44
Список использованных источников (литературы)	45
Приложение А Код библиотеки	47
Приложение Б Руководство пользователя	54
Usb-flash, с разработанным программным В конверте на обороте	
продуктом, пояснительной запиской и обложки	
презентацией.	

					АТДП.09.02.07.20.404.ПЗ			
Изм	Лист	№ документа	Подпись	Дата				
Разраб.		Галанов А.Н.			Разработка программного обеспечения «Создание и обучение моделей распознавания речи»	Литера	Лист	Листов
Проверил		Ситчихин О.В.				У	4	60
						ИСП-20-4		
Н. контр.		Ситчихин О.В.						
Рецензент		Басалаев А.В.						

Введение

Распознавание речи является крайне актуальной задачей в современных условиях стремительного развития технологий искусственного интеллекта и машинного обучения. Технологии распознавания речи находят широкое применение в различных областях: от голосовых ассистентов и систем управления до медицинских и образовательных приложений.

Работы в этой области активно проводятся как отечественными, так и зарубежными учёными и разработчиками. Зарубежные компании, такие как Google, Amazon, и Microsoft, предоставляют облачные сервисы для распознавания речи, предлагая высокую точность и удобство использования. В России также ведутся работы в этом направлении, например, компанией Яндекс.

Целью дипломного проекта является создание программного обеспечения для создания и обучения моделей распознавания речи, которая позволит пользователям самостоятельно обучать модели на собственных данных и использовать их в других системах и приложениях.

Для достижения поставленной цели необходимо выполнить следующие задачи:

- проанализировать предметную область;
- смоделировать структуру программного продукта;
- выбрать программные средства для разработки;
- разработать программный продукт;
- произвести тестирование разработанного продукта;
- выполнить экономические расчёты разработки.

По выполнению задач будет разработано программное обеспечение позволяющее распознавать человеческую речь.

					АТДП.09.02.07.20.404.ПЗ	Лист
						5
Изм	Лист	№ документа	Подпись			

1 Анализ предметной области

Распознавание речи (ASR, Automatic Speech Recognition) — это технология, преобразующая устную речь в текстовый формат. Этот процесс включает захват аудиосигнала, его обработку и интерпретацию для последующего преобразования в текст. Распознавание речи находит широкое применение в различных областях, таких как голосовые помощники, системы управления, медицинские приложения, образовательные платформы, службы автоматического перевода и другие.

На рынке представлены множество известных решений для распознавания речи, среди которых:

- Google Speech-to-Text: облачный сервис от Google, поддерживающий множество языков [15];
- Amazon Transcribe: услуга от Amazon Web Services (AWS), предлагающая функции автоматического распознавания речи с возможностью обработки аудиофайлов и транскрипции в текст [11];
- Microsoft Azure Speech: решение от Microsoft с интеграцией в экосистему Azure [12];
- Яндекс SpeechKit: российский аналог, предоставляющий услуги распознавания речи [6].

Помимо облачных сервисов, существуют настольные и локальные программные решения:

- Dragon NaturallySpeaking: программа от Nuance, предназначенная для преобразования речи в текст, широко используемая в медицинской и юридической сферах;
- Sonix: онлайн-сервис для автоматической транскрипции аудио и видео файлов с поддержкой множества языков;

– Voice2X: отечественная платформа от компании «ЦРТ» (Центр Речевых Технологий), предлагающая автоматический анализ и распознавание речи и совместимая с российскими ОС [16].

Разрабатываемый продукт имеет несколько ключевых преимуществ по сравнению с существующими аналогами:

– локальность: продукт функционирует автономно, без необходимости постоянного подключения к интернету, что обеспечивает высокую степень конфиденциальности данных и независимость от сторонних сервисов;

– возможность создания собственных моделей распознавания речи: пользователи могут создавать и настраивать собственные модели распознавания речи, составлять словари распознаваемых слов и обучать модели на собственных данных, что позволяет адаптировать систему под специфические нужды и требования;

– доступность: продукт разрабатывается для широкого некоммерческого применения, что делает его доступным для широкого круга пользователей.

Большинство программных решений по распознаванию речи являются коммерческими и требуют ежемесячную плату по системе подписки, или платку за каждую минуту аудио. Кроме того, на рынке нет решений, позволяющие легко и интуитивно создавать модели распознавания речи с вручную настроенными параметрами модели в рамках одной программы. Эти обстоятельства выделяют разрабатываемый продукт среди других решений на рынке.

2 Техническое задание

Тип продукта — десктопное программное обеспечение.

1) Основные требования к интерфейсу:

- графический интерфейс: программа должна предоставлять современный графический интерфейс пользователя (GUI), обеспечивая доступ к основным функциям через меню, кнопки и другие элементы управления;

- удобство использования: интерфейс должен быть интуитивно понятным и лёгким в освоении для пользователей с разным уровнем технической подготовки;

- выбор темы интерфейса: интерфейс должен быть адаптирован под светлую и тёмную тему программы.

- навигация: система должна иметь удобную и логичную навигацию, позволяя пользователям быстро находить необходимые функции и разделы;

- отображение процесса обучения и тестирования: интерфейс должен отображать в реальном времени процесс обучения и тестирования модели;

- отображение состояния модели: интерфейс должен отображать имя выбранной модели и её состояние;

2) Карта программного обеспечения.

Программное обеспечение должно обеспечивать доступ ко всем страницам и окнам из любой точки программы. Однако, некоторые страницы и окна могут быть недоступны до выполнения пользователем определённых действий на других страницах или в других окнах. Структура программы представляет собой ступенчатую карту, отображающую последовательность посещения и активации страниц и окон. Переход на следующую ступень открывает доступ ко всем страницам и окнам предыдущих ступеней. На рисунке 1 представлена ступенчатая карта программного обеспечения.

					АТДП.09.02.07.20.404.ПЗ	Лист
						8
Изм	Лист	№ документа	Подпись			



Рисунок 1 Ступенчатая карта программного обеспечения

Первая ступень — стартовая страница. Это страница, на которой пользователь оказывается сразу после запуска программы. С этой страницы пользователь может сразу перейти на вторую ступень.

Вторая ступень — окно выбора модели и страница инициализация аудио. В окне выбора модели, при создании новой модели или выбора модели из списка, открывается доступ к промежуточной, необязательной ступени 2.5, на которой находится окно настройки модели, где уже автоматически установлены значения настроек по умолчанию, и окно словарей модели. После выбора модели и инициализации аудио, появляется доступ к окнам и страницам третьей ступени. Кроме того, если выбранная модель будет уже обучена, то откроется доступ сразу к четвёртой ступени. Если обратно будет выбрана необученная модель, доступ к четвёртой ступени закроется.

Третья ступень — страница обучения модели. После завершения процесса обучения выбранной модели, пользователь может перейти на четвёртую ступень.

Четвёртая ступень — окно транскрипции. В этом окне пользователь может использовав выбранную обученную модель для создания транскрипции к аудио. На этом этапе у пользователя открыты все страница и окна и он может свободно перемещаться из одного места программы в другое.

Требования к функционалу:

- создание модели: пользователь должен иметь возможность создать новую модель распознавания речи;
- выбор модели: пользователь должен иметь возможность выбрать существующую модель из списка, представленного в программе. Список должен содержать информацию о каждом элементе, включая название модели и дату создания и состояние обучения;
- сохранение модели: программа должна предоставлять возможность сохранения текущей модели вместе с её настройками. Сохранение должно происходить в выбранный пользователем каталог;
- удаление модели: пользователь должен иметь возможность удалить выбранную модель из системы;
- выбор пути сохранения: пользователь должен иметь возможность выбрать и изменить каталог для сохранения моделей и загрузки списка существующих моделей;
- настройка параметров модели: программа должна позволять пользователю настраивать параметры модели, такие как параметры алгоритма MFCC и логику обучения DNN;
- создание словарей: пользователь должен иметь возможность создавать новые словари фонем и слов, которые будут использоваться для распознавания речи;

– импортное словари: программа должна предоставлять возможность импорта словарей из файлов формата txt. Импортные словари должны автоматически интегрироваться в систему и быть доступны для использования;

– экспортное словари: пользователь должен иметь возможность экспортировать созданные или отредактированные словари в файл формата txt;

– инициализация аудио: пользователь должен иметь возможность выбора каталогов, содержащих аудио файлы для обучения и тестирования модели;

– обучение модели: программа должна предоставлять функционал для обучения модели на инициализированных аудио файлах;

– распознавание аудио: программа должна предоставлять функционал для выбора аудио файла и его распознавания. Результаты распознавания должны отображаться в реальном времени в виде матрицы вероятностей принадлежности фонемы к кадру и в виде распознанного слова;

– расчёт точности: программа должна предоставлять возможность расчёта точности распознавания каждого слова из словаря модели. После расчёта точности программа должна выводить таблицу, которая включает в себя процент правильно распознанных слов из общего количества для каждого слова;

– воспроизведение аудио: программа должна предоставлять возможность проигрывать выбранный аудио файл;

– создание транскрипции: пользователь должен иметь возможность сгенерировать транскрипцию для выбранного аудио файла. Программа должна автоматически распознавать речь и преобразовывать её в текстовый формат;

– сохранение транскрипции: программа должна предоставлять возможность сохранения сгенерированной транскрипции в файл формата txt.

Требование к интеграции.

Библиотека для использования моделей распознавания речи: должна быть разработана библиотека, позволяющая интегрировать модели распознавания речи в другие системы и приложения.

Документация и руководство пользователя:

– диаграммы: должны быть составлены подробные диаграммы, описывающие функционал и структуру программы, включая архитектуру системы и взаимодействие модулей;

– руководство пользователя: необходимо предоставить руководство пользователя, включающее пошаговые инструкции по установке, настройке и использованию программы;

– комментирование кода: весь код разработанной библиотеки должен быть закомментирован с возможностью автоматической генерации документации.

Техническое задание описывает ключевые аспекты разработки десктопного программного обеспечения. Каким критериям должны соответствовать графический дизайн, структура программы, функционал и документация к программному обеспечению.

					АТДП.09.02.07.20.404.ПЗ	Лист
						12
Изм	Лист	№ документа	Подпись			

3 Моделирование предметной области

Перед разработкой программного обеспечения проводится этап моделирования предметной области и разрабатываемого продукта. На этом этапе выбирается нотация, в которой будут строиться модели и диаграммы. Для моделирования информационных систем используются следующие нотаций:

– UML (Unified Modeling Language) — универсальный язык для моделирования объектов в информационных системах. Включает диаграммы классов, объектов, компонентов, развёртывания, действий, состояний и последовательностей. Широко используется для проектирования и документирования архитектуры программного обеспечения и бизнес-процессов [10];

– BPMN (Business Process Model and Notation) — нотация для моделирования бизнес-процессов, с целью их последующей автоматизации. Создана консорциумом Object Management Group (OMG). Включает элементы, такие как события, задачи, потоки, шлюзы и пулы. Используется для моделирования, анализа и оптимизации бизнес-процессов [2];

– IDEF (Integration DEFinition) — семейство методов моделирования, включающее IDEF0 для функционального моделирования, IDEF1X для моделирования данных и другие. Используется для описания и анализа различных аспектов сложных систем [3];

– SysML (Systems Modeling Language) — язык моделирования систем, основанный на UML. Включает диаграммы для структурных, поведенческих и параметрических аспектов систем. Используется для моделирования сложных систем, включая их аппаратные и программные компоненты [5].

Для моделирования предметной области была выбрана нотация UML (Unified Modeling Language). UML представляет собой стандартный язык моделирования, который широко используется для проектирования и документирования программного обеспечения. Основными преимуществами UML являются:

- универсальность: UML подходит для моделирования различных аспектов систем, включая архитектуру, поведение и взаимодействие компонентов;
- понятность: нотация UML интуитивно понятна и позволяет легко передавать идеи и решения между участниками проекта, включая разработчиков, аналитиков и заказчиков;
- поддержка инструментов: существует множество программных инструментов, поддерживающих UML, что облегчает создание, редактирование и визуализацию моделей;
- стандартизация: UML является международным стандартом (ISO/IEC 19505-1:2012), что обеспечивает его широкое принятие и поддержку в индустрии [13].
- простота: нотация UML не усложнена, как SysML, что делает её удобной для использования в большинстве проектов без необходимости освоения сложных дополнительных элементов.

Для описания разрабатываемого продукта были написаны следующие модели UML:

Диаграмма прецедентов — диаграмма, отображающая взаимодействие пользователей (акторов) с системой. Она показывает, какие функции или услуги система предоставляет пользователям и как пользователи взаимодействуют с системой для достижения своих целей. На рисунке 2 изображена диаграмма прецедентов.

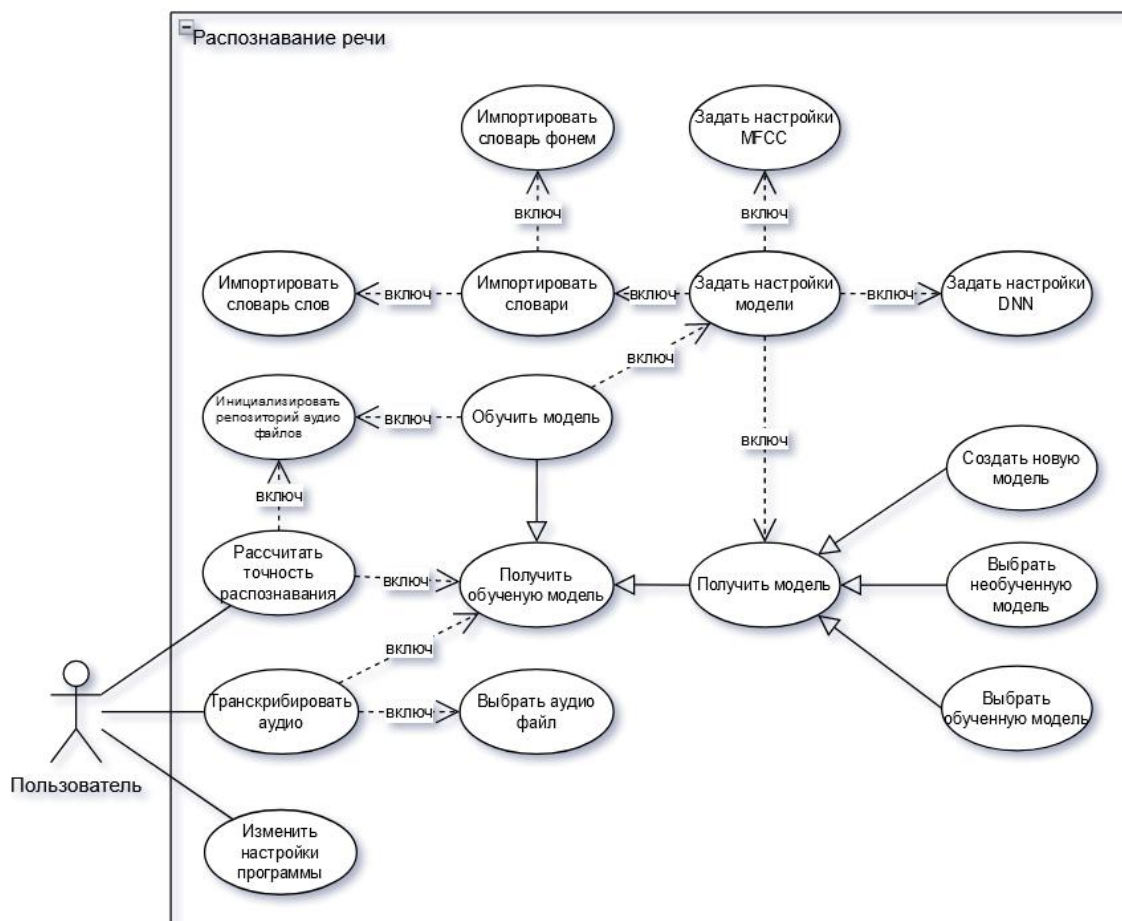


Рисунок 2 — Диаграмма прецедентов

Диаграмма модулей — диаграмма, иллюстрирующая организацию и взаимосвязи между компонентами системы на уровне модулей или подсистем. Она помогает понять архитектуру системы и способы взаимодействия различных частей приложения. На рисунке 3 изображена диаграмма модулей.



Рисунок 3 — Диаграмма модулей

Диаграмма классов — структурная диаграмма, которая показывает статическую структуру системы, описывая классы, их атрибуты, методы и отношения между классами. Эта диаграмма используется для моделирования объектов системы и их взаимосвязей. В качестве диаграммы классов представлена автоматически сгенерированная внутренней системой среды разработки Visual Studio диаграмма классов, на основе разработанного программного продукта. На рисунке 4 изображена диаграмма классов.

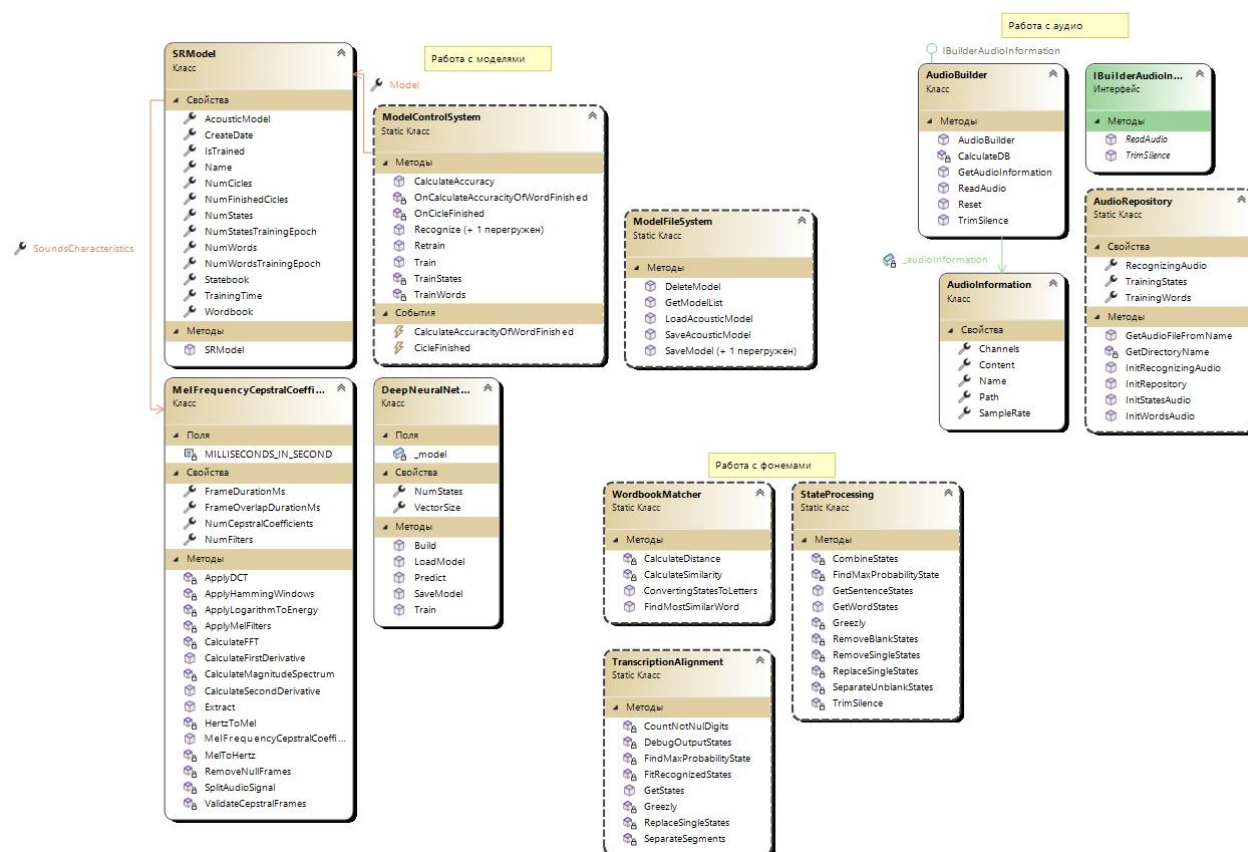


Рисунок 4 — Диаграмма классов

Написанные диаграммы обеспечивают понимание и представление разрабатываемого продукта, охватывая как взаимодействие пользователей с системой, так и внутреннюю структуру и организацию компонентов.

4 Реализация разработки

4.1 Программные средства реализации проекта

Для разработки программного обеспечения, предназначенной для создания и обучения моделей распознавания речи, были отобраны инструменты и технологии, каждая из которых имеет свои преимущества и обоснования выбора.

В качестве основного языка программирования был выбран C# по следующим причинам:

- прикладной язык: C# хорошо подходит для разработки десктопных приложений, обеспечивая высокую производительность и удобство сопровождения подобных программ;
- поддержка библиотек: в C# имеется набор библиотек для работы с аудиофайлами, таких как NAudio и Accord.NET. Эти библиотеки предоставляют функционал для записи, обработки и анализа аудиосигналов, что важно для задачи распознавания речи;
- интеграция с фреймворками машинного обучения: C# легко интегрируется с различными фреймворками для машинного обучения, такими как ML.NET и адаптации TensorFlow, что позволяет использовать современные алгоритмы глубокого обучения непосредственно в проекте.

Visual Studio была выбрана в качестве среды разработки по следующим причинам:

- поддержка десктопных программ: Visual Studio отлично подходит для создания десктопных приложений с поддержкой сложных и интуитивно понятных пользовательских интерфейсов;
- удобство и функциональность: Visual Studio предоставляет мощный и интуитивно понятный интерфейс, который упрощает процесс написания,

отладки и тестирования кода. Это повышает общую производительность разработки и сокращает время на устранение ошибок;

- поддержка инструментов: среда разработки включает множество встроенных инструментов для анализа кода и управления проектом, что упрощает работу в разработке;

- расширяемость: возможность добавления дополнительных расширений и плагинов позволяет адаптировать Visual Studio под специфические нужды проекта, обеспечивая гибкость и расширяемость среды разработки.

Для реализации машинного обучения был использован TensorFlow. Основные причины выбора включают:

- современные методы глубокого обучения: TensorFlow предоставляет доступ к передовым методам и алгоритмам глубокого обучения, что позволяет создавать высокоэффективные модели распознавания речи;

- мощность и производительность: TensorFlow оптимизирован для работы на различных платформах, включая локальные вычислительные мощности и облачные сервисы, что обеспечивает высокую производительность при обучении и использовании моделей;

- широкая поддержка и сообщество: TensorFlow имеет обширную документацию и активное сообщество разработчиков, что упрощает процесс обучения и решения возникающих проблем.

Для записи и предварительной обработки аудиофайлов использовалась программа Audacity, которая была выбрана по следующим причинам:

- простота использования: Audacity предоставляет интуитивно понятный интерфейс, который позволяет легко записывать, редактировать и экспортировать аудиофайлы, необходимые для обучения и тестирования моделей;

- функциональность: программа поддерживает широкий спектр функций для обработки аудио, включая фильтрацию шумов, нормализацию

громкости и разбиение на фреймы, что критически важно для подготовки данных;

– доступность: Audacity является бесплатной и с открытым исходным кодом, что делает её доступной для использования без дополнительных затрат, а также позволяет вносить изменения в код при необходимости.

Все выбранные программные средства разработки использованы для создания единого программного обеспечения выполняющие требования в соответствии с техническим заданием.

4.2 Структура информационной системы

Основной задачей программного обеспечения и разработанной библиотеки является распознавание человеческой речи. Для решения этой задачи создан модуль распознавания речи, включающий все необходимые алгоритмы обработки аудио и обучения модели. Каждый алгоритм размещён в отдельном модуле и тщательно отобран и протестирован среди множества других возможных алгоритмов.

Алгоритм распознавания речи включает несколько ключевых этапов: последовательную обработку аудиосигнала, извлечение характеристик, классификацию звуков в фонемы и сопоставление их со словами из словаря. На рисунке 5 изображена структура распознавания речи.

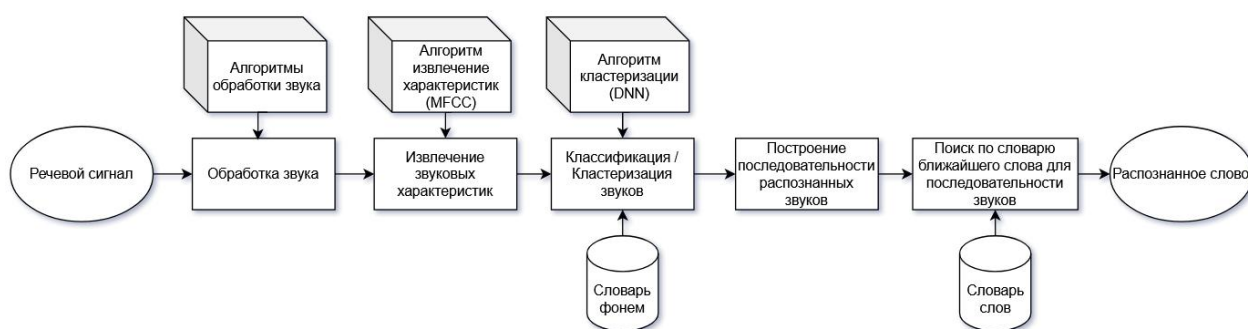


Рисунок 5 — Структура распознавания речи

Алгоритм распознавание речи включает следующие этапы:

- сбор и предварительная обработка аудиоданных: на этом этапе осуществляется запись или загрузка аудио файла, его нормализация и фильтрация для удаления шума и улучшения качества сигнала;

- извлечение звуковых характеристик (MFCC): извлечение характеристик из аудиосигнала производится с использованием коэффициентов мел-кепстральных частот (MFCC). MFCC являются стандартным методом для представления аудиосигнала в более компактной и информативной форме. Они моделируют человеческое восприятие звуков, что позволяет эффективно анализировать и распознавать речь;

- кластеризация звуков (DNN): после извлечения характеристик, обработанные данные передаются в глубокую нейронную сеть (DNN). DNN используется для классификации звуков в отдельные фонемы. Фонема — это минимальная звуковая единица языка, которая может изменить смысл слова (например, «м», «н», «п»). Фонемы служат основными распознаваемыми элементами в алгоритме [8];

- построение последовательности фонем: на этом этапе из последовательности кластеризованных звуковых характеристик строится последовательность фонем, которая соответствует произнесённой речи;

- работа со словарём: словарь содержит все распознаваемые слова модели. Каждое слово представлено как комбинация фонем. При распознавании речи, полученная комбинация фонем сравнивается с комбинациями, хранящимися в словаре. Используя алгоритмы сопоставления, определяется наиболее вероятное слово, которое было произнесено;

- вывод результата: на основе сопоставления последовательности фонем со словарём, система выводит распознанное слово или фразу.

Существует множество алгоритмов извлечения звуковых характеристик, каждый из которых имеет свои особенности и области применения. Основные из них включают:

					АТДП.09.02.07.20.404.ПЗ	Лист
						20
Изм	Лист	№ документа	Подпись			

– Linear Predictive Coding (LPC): этот метод анализирует звуковые сигналы и прогнозирует будущее значение сигнала на основе его прошлых значений. LPC является мощным методом для параметрического представления речевого сигнала и используется в различных приложениях, включая кодирование речи и распознавание речи [14];

– Perceptual Linear Prediction (PLP): PLP улучшает LPC путём моделирования человеческого слухового восприятия. Этот метод использует перцептивные шкалы частоты и громкости для более точного моделирования звуковых характеристик [14];

– Gammatone Frequency Cepstral Coefficients (GFCC): GFCC основаны на модели гаматоновых фильтров, которые имитируют свойства человеческого уха. Этот метод показывает высокую точность в условиях с шумом;

– Mel-Frequency Cepstral Coefficients (MFCC): MFCC основываются на шкале мел, которая имитирует человеческое восприятие частоты. Этот метод преобразует звуковой сигнал в мел-частотное пространство, что позволяет эффективно анализировать звуковые характеристики [1, 4];

– Short-Time Fourier Transform (STFT): STFT разлагает сигнал на временные окна и вычисляет спектр каждого окна, что позволяет получить временно-частотное представление сигнала;

– Wavelet Transform (WT): WT разлагает сигнал на компоненты с разными частотами и временными разрешениями, что позволяет получить многомасштабное представление сигнала.

MFCC был выбран в качестве основного алгоритма извлечения звуковых характеристик для системы распознавания речи по следующим причинам:

– преимущество в моделировании человеческого слуха: MFCC использует шкалу мел, которая более точно отражает человеческое восприятие частоты. Это делает MFCC особенно эффективным для анализа

речевых сигналов, так как он учитывает особенности восприятия звуков человеком;

- широкое распространение и доказанная эффективность: MFCC является одним из самых популярных и хорошо изученных методов для извлечения звуковых характеристик. Он широко используется в системах распознавания речи и показал свою высокую эффективность в различных условиях, включая шумные среды;

- относительная простота и скорость вычислений: несмотря на свою мощность, алгоритм MFCC относительно прост в реализации и требует умеренных вычислительных ресурсов. Это позволяет его использовать в реальных приложениях, где важны производительность и быстрота обработки;

- совместимость с различными моделями машинного обучения: MFCC предоставляет компактное и информативное представление звукового сигнала, что делает его удобным для использования в сочетании с различными моделями машинного обучения, включая глубокие нейронные сети (DNN).

В совокупности, эти преимущества делают MFCC оптимальным выбором для системы распознавания речи, обеспечивая высокую точность и эффективность при обработке звуковых сигналов.

Несмотря на текущий выбор MFCC для извлечения звуковых характеристик, в дальнейшем планируется внедрить в систему поддержку других алгоритмов, таких как Perceptual Linear Predictive (PLP) и Gammatone Frequency Cepstral Coefficients (GFCC). PLP может улучшить распознавание, учитывая особенности человеческого восприятия звука, а GFCC, основанный на моделировании человеческого слуха, может повысить точность и устойчивость системы в условиях шума. Это расширение алгоритмической базы обеспечит более гибкую и мощную систему распознавания речи, способную адаптироваться к разнообразным условиям и улучшать качество распознавания.

Существует несколько алгоритмов кластеризации, которые могут быть использованы для анализа и распознавания звуковых сигналов. Основные из них включают:

- скрытые Марковские модели (Hidden Markov Models, HMM): HMM используются для моделирования последовательностей, таких как речевые сигналы. Они основаны на вероятностных процессах и часто применяются в распознавании речи, так как могут эффективно моделировать временные зависимости;

- K-Means++: Этот алгоритм кластеризации делит данные на K кластеров, минимизируя внутрикластерное расстояние. K-Means++ улучшает стандартный K-Means за счет более разумного выбора начальных центров кластеров, что позволяет улучшить конечный результат;

- Гауссовы смеси (Gaussian Mixture Models, GMM): GMM представляют данные как комбинацию нескольких гауссовых распределений. Этот метод часто используется для моделирования сложных распределений данных и имеет хорошие результаты в задачах кластеризации и классификации;

- глубокие нейронные сети (Deep Neural Networks, DNN): DNN могут моделировать сложные нелинейные зависимости и широко используются в различных задачах машинного обучения, включая распознавание речи. Они способны обучаться на больших объемах данных и показывают высокую точность в условиях с шумом.

Ранее в разрабатываемой программе распознавания речи использовались такие технологии как HMM, K-Means++ и GMM. Эти методы показали свою неэффективность при распознавании отдельных фонем и подходили лишь для распознавания целых слов. В отличие от них, DNN были выбраны по следующим причинам:

- высокая точность распознавания фонем: показали значительно лучшие результаты при распознавании отдельных фонем по сравнению с

НММ, K-Means++ и GMM. Это связано с их способностью моделировать сложные нелинейные зависимости и учитывать контекстные особенности звуков;

- гибкость и адаптивность: DNN могут быть обучены на больших объёмах данных и способны адаптироваться к различным условиям, таким как наличие шума или изменение акцентов. Это делает их более универсальными и надёжными в практическом применении;

- интеграция с библиотеками и инструментами: DNN легко интегрируются с популярными библиотеками машинного обучения, такими как TensorFlow и PyTorch, что упрощает процесс разработки и тестирования моделей. Это также обеспечивает доступ к мощным инструментам для визуализации и анализа данных.

Эти преимущества делают DNN оптимальным выбором для системы распознавания речи, обеспечивая высокую точность, адаптивность и эффективность при обработке звуковых сигналов [9].

Несмотря на текущий выбор DNN для кластеризации звуковых характеристик, в дальнейшем планируется внедрение в систему поддержки других алгоритмов кластеризации, основанных на нейронных сетях, таких как сверточные нейронные сети (CNN) и рекуррентные нейронные сети (RNN). CNN могут улучшить распознавание за счёт эффективного выделения пространственных признаков в звуковых данных, а RNN, благодаря своей способности учитывать временные зависимости, могут повысить точность и контекстуальную осведомлённость системы. Это расширение алгоритмической базы обеспечит более гибкую и мощную систему распознавания речи, способную адаптироваться к разнообразным условиям и улучшать качество распознавания.

При разработке системы распознавания речи существует несколько подходов к выбору базовых единиц для распознавания. Эти единицы могут быть словами, фонемами или частями слов и слогами. Фонема – это минимальная звуковая единица языка, способная изменять значение слова [7]. В контексте распознавания речи фонемы являются базовыми элементами, которые классифицируются и выстраиваются в слова. Самым неэффективным подходом является использование целых слов из-за их большого количества и вариативности. В то время как использование частей слов и слогов является наиболее эффективным методом, текущая система распознавания речи базируется на фонемах по причине того, что фонемы являются минимальными звуковыми единицами языка, и их количество в любом языке ограничено и относительно невелико. Это делает их удобными для обучения и обработки. В отличие от слов, которые могут исчисляться тысячами, количество фонем обычно составляет несколько десятков. Например, в русском языке существует около 40, 50, 60 фонем в зависимости от фонетической школы. Такое ограниченное количество позволяет более эффективно обучать модели на аудиоданных, так как требуется меньше данных для охвата всех возможных фонем.

Таким образом, выбор фонем как базовых единиц для распознавания речи в текущей системе обусловлен их ограниченным количеством и относительной лёгкостью обучения, что обеспечивает высокую точность и эффективность системы.

Процесс обучения модели распознавания речи включает два основных этапа.

Первый этап — обучение модели на изолированных фонемах. На этом этапе каждая фонема обучается индивидуально, используя соответствующие аудиофайлы. Этот процесс позволяет модели овладеть базовыми акустическими особенностями каждой фонемы, что является фундаментом для дальнейшего обучения и повышения точности распознавания речи.

					АТДП.09.02.07.20.404.ПЗ	Лист
						25
Изм	Лист	№ документа	Подпись			

Второй этап — обучение модели в контексте слов, учитывая взаимосвязи между фонемами в словах. Этот этап реализуется с использованием двуступенчатого подхода. Первая ступень — предварительно обученная на изолированных фонемах модель используется для распознавания аудиоданных, содержащих слова. Затем проводится выравнивание транскрипций, что обеспечивает соответствие между произнесённым словом и распознанными фонемами. Вторая ступень — модель обучается на основе аудиоданных с уже выровненными транскрипциями. Благодаря итеративному характеру этого процесса, каждая итерация приводит к более точным транскрипциям и, соответственно, более точному обучению модели [7].

4.3 Описание программных модулей

Разработанная система включает несколько основных окон и страниц для взаимодействия с пользователем:

- Guide страница: сводка о назначении окон и страниц, обеспечивающая пользователя общей информацией о функционале системы и взаимодействия с ней;
- ModelList окно: интерфейс для выбора существующих моделей или создания новых. Пользователь может просматривать список доступных моделей, создавать новые или загружать существующие;
- ModelSettings окно: форма для настройки параметров модели, включая алгоритмы MFCC и DNN. Пользователь может установить значения для различных параметров, необходимых для оптимального функционирования модели;
- ModelDictionaries окно: интерфейс для формирования и редактирования словарей фонем и слов. Это окно позволяет пользователю создавать и импортировать словари, которые будут использоваться при распознавании речи;

– ModelTrain страница: страница для обучения и тестирования модели. Пользователь может запустить процесс обучения модели, проверить точность распознавания модели для отдельных аудио файлов и рассчитать общую точность распознавания слов из словаря;

– Transcription окно: окно для генерирования транскрипций для аудио файла. Пользователь может выбрать аудио файл, после чего модель распознает речь и сгенерирует транскрипцию, Которую можно будет сохранить в текстовом файле;

– AudioRepositories страница: страница для инициализации аудио данных в программе. Пользователь может загружать и управлять аудио файлами, которые будут использоваться в обучении и тестировании моделей.

Основные алгоритмы, используемые в системе, включают:

– DeepNeuralNetworks (Глубокие нейронные сети): класс представляет собой модель глубокой нейронной сети, специально настроенную для анализа характеристик звука. Методы включают в себя построение модели глубокой нейронной сети с определённой архитектурой, обучение сети с использованием предоставленных характеристик звука и состояний, предсказание состояний на основе характеристик звука, сохранение обученной модели и загрузку обученной модели из файла;

– MelFrequencyCepstralCoefficients (Мел-частотные кепстральные коэффициенты): класс предоставляет методы для вычисления MFCC для заданного аудиосигнала. Методы включают в себя извлечение признаков MFCC из предоставленной аудиоинформации;

– ModelControlSystem (Система управления моделью): класс предоставляет методы для управления обучением, переобучением и распознаванием речи с использованием SRModel. Методы включают в себя обучение модели, переобучение модели, распознавание речи, тренировку акустической модели на уровне состояний и слов, вычисление точности модели для каждого слова в словаре [7];

– ModelFileSystem (Файловая система модели): класс предоставляет методы для сохранения, загрузки и удаления экземпляров SRModel из файловой системы. Методы включают в себя сохранение текущей модели, удаление указанной модели, получение списка всех моделей, сохранение акустической модели текущего экземпляра SRModel и загрузку акустической модели текущего экземпляра SRModel из файловой системы;

– SRModel (Модель распознавания речи): класс представляет собой модель распознавания речи, включающую акустические и языковые компоненты. В классе предусмотрены свойства, отражающие информацию о модели, а также метод инициализации нового экземпляра модели с указанием MFCC и глубокой нейронной сети в качестве параметров;

– StateProcessing (Обработка состояний): класс предоставляет методы для обработки вероятностей состояний и извлечения распознанных состояний. Методы включают в себя обработку вероятностей состояний, выделение распознанных состояний, поиск состояния с максимальной вероятностью, замену одиночных состояний на окружающее состояние, удаление одиночных состояний, не предшествующих или не следующих за одним и тем же состоянием, удаление пустых состояний и объединение смежных дублирующихся состояний;

– TranscriptionAlignment (Выравнивание транскрипции): класс предоставляет методы для выравнивания транскрипций. Методы включают в себя получение выровненных состояний на основе вероятностей состояний и целевых состояний, извлечение распознанных состояний, поиск состояния с максимальной вероятностью, замену одиночных состояний на окружающее состояние, подгонку распознанных состояний к целевым состояниям и разделение распознанных состояний на сегменты;

– WordbookMatcher (Поиск совпадений в словаре): класс предоставляет методы для поиска наиболее похожего слова из словаря на основе расстояния Левенштейна. Методы включают в себя поиск наиболее похожего слова из

словаря для заданного слова, расчёт сходства между двумя словами с использованием расстояния Левенштейна и расчёт расстояния Левенштейна между двумя словами;

– AudioInformation (Информация об аудио данных): класс представляет информацию об аудио файле, такую как путь к файлу, имя файла, количество каналов, частота дискретизации и содержимое аудио в виде массива образцов;

– AudioRepository (Репозиторий аудио): класс предоставляет методы для инициализации и управления репозиторием аудио. Методы включают в себя инициализацию репозитория для тренировки состояний, тренировки слов и распознавания аудио, получение имени директории из указанного пути и получение аудио файла из указанного имени файла и словаря аудио информации;

– IBuilderAudioInformation (Интерфейс построения информации об аудио): интерфейс определяет методы для чтения аудио данных и обрезки тишины;

– AudioBuilder (Построитель аудио): класс реализует интерфейс IBuilderAudioInformation и предоставляет функциональность для чтения аудио из файла и обрезки тишины. Методы включают в себя чтение аудио из файла и обрезку тишины с помощью заданного порога децибел.

Эти классы и методы составляют основу системы распознавания речи, предоставляя функциональность для анализа звуковых данных, обучения моделей и управления ими. Код классов предоставлен в Приложении А.

5 Тестирование разработанного продукта

5.1 Анализ и выбор методов тестирования

При разработке программного обеспечения «создания и обучения моделей распознавания речи» был проведён комплексный процесс тестирования и отладки. В рамках этого процесса использовались следующие методы тестирования:

- функциональное тестирование: использовалось для проверки общей работоспособности системы. Оно включало проверку всех функциональных возможностей, таких как создание новых моделей, настройка параметров, распознавание речи и сохранение обученных моделей. Тестирование охватывало сценарии, имитирующие действия конечных пользователей, чтобы убедиться в корректной работе системы в реальных условиях;

- модульное тестирование с помощью Unit тестов: были написаны Unit тесты для классов StateProcessing, TranscriptionAlignment и WordbookMatcher. Эти тесты обеспечивали проверку корректности работы ключевых алгоритмов и их взаимодействий;

- использование системы автоматического тестирования кода: применялась автоматизированная система тестирования SonarLint, проводившая статический анализ кода. SonarLint выявлял потенциальные ошибки, проблемы с производительностью и уязвимости безопасности, что позволило устранить множество дефектов на ранних стадиях разработки;

- разработка и использование собственных инструментов тестирования точности распознавания речи: разработанные инструменты были включены в функционал программы и её интерфейс, что обеспечивало проверку верности и точности работы моделей распознавания речи.

5.2 Результаты тестирования

Функциональное тестирование и отладка позволило выявить и устранить множество ошибок в процессе разработки системы. Здесь предоставлен некоторых из них:

- неверное использование многопоточности при обучении модели, что вызывало преждевременное окончание работы;
- использование первых и вторых производных мел-частотно кепстральных коэффициентов, что втроекратно увеличивало время обучения модели и не улучшало её точность;
- некорректное сохранение настроек обученной модели, приводившее к невозможности её повторного использования;
- неверная инициализация репозитория аудиофайлов, что делало невозможным использование аудиофайлов для тестирования обученной модели;
- множество ошибок пользовательского интерфейса;
- регулярные ошибки в коде.

Модульное тестирование выявляло ошибки в алгоритмах и в логике реализации функций.. Предоставлены некоторые из таких ошибок:

- неверное выравнивание транскрипции, приводившее к расхождению двуступенчатого алгоритма обучения модели;
- ошибки в обработке состояний, что напрямую влияло на точность распознавания;
- неэффективность алгоритма поиска ближайшего слова в словаре для различных комбинаций слов, где ближайшее слово определялось неверно.

После устранения этих ошибок все Unit тесты регулярно запускались после внесения изменений в соответствующие части кода, что поддерживало верность и корректность выполнения алгоритмов.

На рисунке 6 изображены тесты класса StateProcessingTests.

StateProcessingTests (8)	4 мс
CombineStates_CombinesCorrectly	< 1 мс
GetSentenceStates_NullInput_ThrowsArgumentNullException	< 1 мс
GetSentenceStates_ValidInput_ReturnsExpectedStates	< 1 мс
GetWordStates_NullInput_ThrowsArgumentNullException	< 1 мс
GetWordStates_ValidInput_ReturnsExpectedStates	< 1 мс
RemoveBlankStates_RemovesCorrectly	4 мс
RemoveSingleStates_RemovesCorrectly	< 1 мс
ReplaceSingleStates_ReplacesCorrectly	< 1 мс

Рисунок 6 — Тесты класса StateProcessingTests

На рисунке 7 изображены тесты класса TranscriptionAlignmentTests.

TranscriptionAlignmentTests (7)	4 мс
FindMaxProbabilityState_ReturnsCorrectIndex	< 1 мс
FitRecognizedStates_FitsCorrectly	1 мс
GetStates_NullStateProbabilities_ThrowsArgumentNullException	< 1 мс
GetStates_NullTargetStates_ThrowsArgumentNullException	2 мс
GetStates_ValidInput_ReturnsAlignedStates	1 мс
ReplaceSingleStates_ReplacesCorrectly	< 1 мс
SeparateSegments_SeparatesCorrectly	< 1 мс

Рисунок 7 — Тесты класса TranscriptionAlignmentTests

На рисунке 8 изображены тесты класса WordbookMatcherTests.

WordbookMatcherTests (9)	3 мс
CalculateDistance_NullOrEmptyWords_ThrowsArgumentException	< 1 мс
CalculateDistance_ValidInput_ReturnsCorrectDistance	1 мс
CalculateSimilarity_ValidInput_ReturnsCorrectSimilarity	< 1 мс
ConvertingStatesToLetters_EmptyRecognizedStates_ReturnsEmptyString	< 1 мс
ConvertingStatesToLetters_ValidInput_ReturnsCorrectString	2 мс
FindMostSimilarWord_EmptyWordbook_ThrowsArgumentNullException	< 1 мс
FindMostSimilarWord_NullWord_ThrowsArgumentNullException	< 1 мс
FindMostSimilarWord_NullWordbook_ThrowsArgumentNullException	< 1 мс
FindMostSimilarWord_ValidInput_ReturnsMostSimilarWord	< 1 мс

Рисунок 8 — Тесты класса WordbookMatcherTests

Собственные инструменты тестирования позволили протестировать работоспособность всех разработанных алгоритмов распознавания. Инструменты позволяют выбирать аудиофайлы и строить для них матрицу вероятностей, где по одной оси располагаются кадры, по другой распознанные фонемы, и где каждая ячейка содержит вес принадлежности фонемы к кадру. Также был разработан алгоритм расчёта точности распознавания слов, находящихся в словаре тестируемой модели. На рисунке 9 изображена матрица вероятностей.

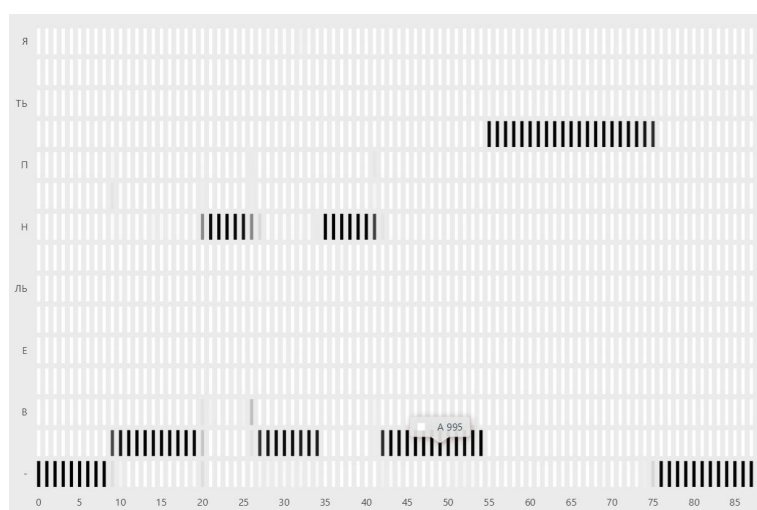


Рисунок 9 — Матрица вероятностей

На рисунке 10 изображена таблица точности распознавания словаря модели.

Accuracy	
Calculate	
100	АНАНАС
100	АПОДИЯ
100	ВОСЕМЬ
100	ДВА
100	ДЕВА
100	ДЕВЯТЬ
100	ОДИН
100	ОПАД
100	ПОШИВ
95	ПЯТЬ
100	СЕМЬ
75	ШАНС
95	ШЕСТЬ
Clear	

Рисунок 10 — Таблица точности распознавания словаря

Кроме тестирования, было проведено профилирование производительности работы программы. На рисунке 11 изображены результаты профилирования.

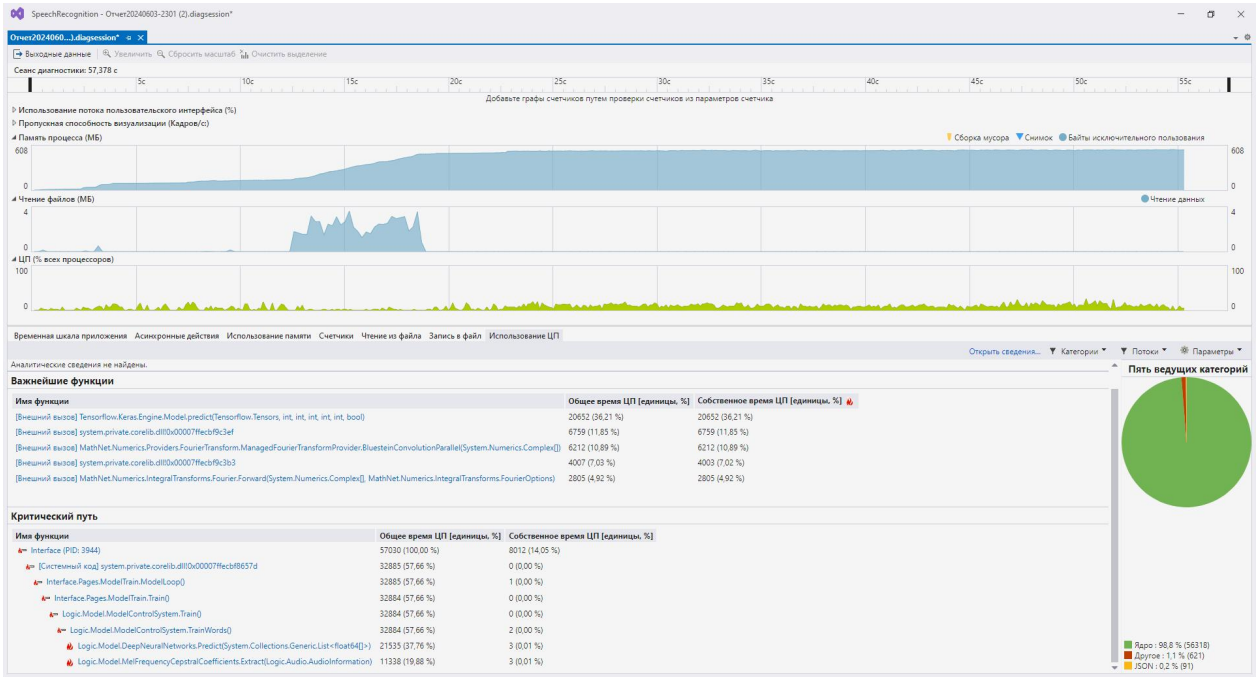


Рисунок 11 — Отчёт профилирования производительности

В отчёте профилирования на графике чтения файлов наблюдаются значительные пики. Это связано с процессом чтения и инициализации аудиофайлов, которые загружаются в оперативную память для дальнейшего использования в обучении и тестировании моделей. В дальнейшем использовании программы, включая наиболее ресурсоёмкий процесс обучения моделей, не наблюдается существенных скачков в производительности.

В результате проведённого тестирования и отладки была подтверждена надёжность работы разработанной системы. Однако, процесс тестирования остаётся непрерывным, и новые тесты будут добавляться по мере развития системы и внедрения новых функций, для поддержания качества продукта и соответствии требованиям пользователей.

6 Экономическое обоснование разработки

На современном рынке существует множество известных решений для распознавания речи, таких как Google Speech-to-Text, Amazon Transcribe, Microsoft Azure Speech, Яндекс SpeechKit и другие. Каждое из этих решений обладает определёнными преимуществами и недостатками. Разрабатываемый продукт представляет собой решение, имеющее ряд преимуществ, благодаря которым превосходит свои аналоги. Этими преимуществами являются:

- автономность: продукт функционирует локально, что исключает необходимость постоянного подключения к интернету. Это существенно повышает уровень конфиденциальности и безопасности данных, что является критически важным для многих пользователей и организаций;

- гибкость обучения: продукт предоставляет возможность обучать модели на собственных данных пользователя. Это позволяет адаптировать систему под специфические нужды и требования различных пользователей, что значительно повышает её универсальность и применимость в различных контекстах;

- интеграция: продукт оснащён библиотекой для лёгкой интеграции функционала распознавания речи в другие информационные системы. Это упрощает процесс внедрения и использования продукта в существующих ИТ-инфраструктурах.

Также в отличие от аналогов, продукт предоставляется на безвозмездной основе, что исключает необходимость подписки или иных регулярных платежей, характерных для коммерческих систем распознавания речи. Это делает его экономически выгодным выбором для организаций и частных пользователей.

Программисту, участвовавшему в разработке программного продукта, начислялся минимальный размер оплаты труда в сумме 19242 рубля в месяц. Выбор минимального размера оплаты труда связан с тем фактом, что продукт разрабатывался студентом и создавался для широкого рынка без коммерческих целей. В таблице 1 отображён расчёт заработной платы.

Таблица 1 — Расчёт заработной платы

Этапы работ	Наименование видов работ	Исполнитель		Часовая ставка	Трудоёмкость		Размер заработной платы
		Должность	Количество		дней	часов	
Начальный	Анализ предметной области и формулирование требований	программист	1	110	7	56	6160
Проектирование	Разработка архитектуры и структуры программы	программист	1	110	7	56	6160
Разработка	Разработка каждого компонента и кодирование на языке программирования	программист	1	110	30	240	26400
Тестирование	Тестирование компонентов и комплексное тестирование программы	программист	1	110	4	32	3520
Заключительный	Коррекция программы	программист	1	110	7	56	6160
Итого			5	550	55	440	48400

Время, затраченное на разработку составило 440 час или 2,5 месяца. Где в месяце 175 рабочих часов. В дальнейших расчётах затраченное время на разработку было принято за 2,5 месяца.

Вместе с заработной платной, рассчитан фонд оплаты труда. В таблице 2 отображён расчёт фонд оплаты труда за месяц.

Таблица 2 — Расчёт фонда оплаты труда за месяц

Подразделение (исполнитель)	Сумма заработной платы с учетом надбавок (основная зарплата)	Дополнительная зарплата (премии и доплаты)	Коэффициент, учитывающий природные условия (УК)	Итого фонд заработной платы (ФЗП)	Начисления на зарплату	Всего Фонд оплаты труда (ФОТ)
		20%	15%		30,20%	
Б	1	2	3	4	5	6
Программист	48400	9680	1452	59532	17978,66	77510,66
Всего	48400	9680	1452	59532	17978,66	77510,66

Фонд оплаты труда составил 77 510 рублей и 66 копеек.

После расчёта фонда оплаты труда, рассчитывалась амортизация оборудования, которое было использовано во время разработки программного продукта.

Амортизация — это процесс постепенного переноса стоимости долгосрочных активов (например, зданий, оборудования, машин) на издержки производства или обращения в течение их полезного срока службы. Этот процесс позволяет равномерно распределить затраты на приобретение актива на протяжении нескольких отчётных периодов. В таблице 3 отображён расчёт годовой суммы амортизационных отчислений на оборудование.

Таблица 3 — Расчёт годовой суммы амортизационных отчислений

Элементы основных фондов	Кол- во, ед.	СПИ	Первоначальная стоимость основных фондов, руб. за ед.	Общая стоимость основных фондов, руб.	Норма амортиз ации годовая, %%	Амортизационные отчисления, рублей	
						В год, мес.	На разработ ку, мес.
						12	2,5
Компьютерный стол	1	0	8 000,00	8 000,00	10,00	800,00	166,67
Стул офисный	1	3	3 000,00	3 000,00	33,33	1 000,00	208,33
Компьютер	1	5	110 000,00	110 000,00	20,00	22 000,00	4 583,33
Звукозаписываю щее устройство (Микрофон)	1	7	4 000,00	4 000,00	14,29	571,43	119,05
Устройство вывода звука (Наушники)	1	7	4 000,00	4 000,00	14,29	571,43	119,05
Маршрутизатор	1	7	2 000,00	2 000,00	14,29	285,71	59,52
Всего	6		131 000,00	131 000,00	106,19	25 228,57	5 255,95

Амортизация рассчитывалась по формуле (1).

$$N_a = 1 / \text{СПИ} \quad (1)$$

где:

- N – годовая норма амортизации;
- СПИ – срок полезного использования (СПИ) объекта ОС в годах.

Общая амортизация всего оборудования составила 7989,05 руб.

Для использования оборудования и разработки программного продукта использовалась электроэнергия, питающее всё используемое оборудование. В Таблице 4 отображён расчёт затрат на электроэнергию, потребляемую оборудованием во время разработки программного продукта.

Таблица 4 — Расчёт затрат на электроэнергию на разработку

Наименование оборудования	Кол-во оборудования, ед.	Потребляемая мощность в час, кВт	Время работы оборудования, час	Стоимость 1 кВт/час, руб.	Сумма затрат на разработку, руб.
Освещение (кол-во ламп*мощность ламп)	2	0,04	340,00	5,05	52,52
Компьютер	1	0,30	680,00	5,05	1030,20
Звукозаписывающее устройство (Микрофон)	1	0,01	568,00	5,05	180,54
Устройство вывода звука (Наушники)	1	0,01	568,00	5,05	4,69
Маршрутизатор	1	0,01	680,00	5,05	11,25
Всего	6	0,36	2836,00	25,25	1279,20

При разработке продукта в качестве расходного материала использовалась только флешка. В таблице 5 отображён расчёт затрат на расходные материалы.

Таблица 5 — Расчёт затрат на расходные материалы на разработку

Наименование оборудования	Кол-во необходимого расходного материала на разработку, ед.	Стоимость ед, расходного материала, руб.	Сумма затрат на расходные материалы на разработку, руб.
Флешка	1	350	350

В качестве дополнительного носителя информации и пространства для проектирования и структурирования информации, использовалась бумага и записывающее устройство ручка. В таблице 6 отображён расчёт прочих расходов.

Таблица 6 — Прочие расходы

Наименование оборудования	Ед.изм.	Количество на разработку	Цена за ед., руб.	Сумма затрат на разработку, руб.
Бумаг	Пач.	1	291	291
Ручки, карандаши	Шт.	1	63	63
Всего		2	354	354

В рамках разработки программного обеспечения не проводились расчёты для накладных расходов, аренды помещения и затрат на хостинг. Эти виды затрат не предусматривались и не являлись необходимыми на этапе создания продукта. Разработка велась с использованием имеющихся ресурсов и без привлечения дополнительных инфраструктурных вложений. Таким образом, итоговая калькуляция затрат на разработку исключает данные этих расходов.

Итоговая калькуляция затрат на разработку программного обеспечения суммирует результаты всех предыдущих расчётов и содержит себестоимость разработки, планируемый уровень рентабельности, всего затрат, НДС и отпускную цену. В таблице 7 отображена плановая калькуляция затрат на разработку продукта.

Таблица 7 — Плановая калькуляция затрат на разработку продукта

№№ пп	Наименование статьи калькуляции	Коэффициент	Сумма в месяц, руб.
1	Основная заработная плата		48400,00
2	Дополнительная заработная плата		9680,00
3	Уральский коэффициент	15%	1452,00
4	Итого фонд заработной платы		59532,00
5	Отчисления в страховые фонды	30,20%	17978,66
6	Итого фонд оплаты труда		77510,66
7	Амортизация		5255,95
8	Затраты на материалы, включая расходные материалы		1539,82
9	Аренда помещения		0,00
10	Затраты на доменное имя и хостинг (размещение в сети интернет)		0,00
11	Прочие расходы (канцелярские и др.)		354,00
12	Накладные расходы		0,00
13	Итого себестоимость (С)		84660,43
14	Планируемы уровень рентабельности	20%	16932,09
15	Всего затрат		101592,52
16	НДС	20%	20318,50
17	Отпускная цена		121911,02

Себестоимость — это сумма всех затрат, понесённых предприятием на производство и реализацию продукции, оказание услуг или выполнение работ.

Цена рассчитана по формуле (2).

$$\begin{aligned}
 & \text{Итоговый фонд оплаты труда} + \text{Амортизация} \\
 & + \text{Затраты на материалы, включая расходные} \\
 & + \text{Аренда помещений} \\
 & + \text{Затраты на доменное имя и хостинг} \\
 & + \text{Прочие расходы} + \text{Накладные расходы}
 \end{aligned} \quad (2)$$

Себестоимость продукта составила 84660,43 руб.

Для оценки финансовой эффективности разработки сравним разработанный продукт с тарифами аналогичных услуг, предоставляемых другими компаниями. Тарифы таких компаний на 01.06.24 следующие:

- Google Speech-to-Text: \$0.024 за минуту аудио (2,15 руб.) [15];
- Amazon Transcribe: \$0.024 за минуту аудио (2,15 руб.) [11];
- Microsoft Azure Speech: \$0.017 за минуту аудио (1,52 руб.) [12];
- Яндекс SpeechKit: 0,96 руб. за минуту аудио [6].

Чтобы определить необходимое количество аудио для окупаемости себестоимости разработки в размере 87693,15 руб. при цене Яндекс SpeechKit 0,96 руб. за минуту аудио, применяется формула (3).

$$\frac{84660,43 \text{ руб.}}{0,96 \text{ руб./мин}} = 88\,188 \text{ мин} \quad (3)$$

Таким образом, с учётом минимальной заработной платы, для окупаемости затрат на разработку необходимо обработать приблизительно 88188 минут аудио, что эквивалентно 1470 часам или 61 суткам.

Внедрение разработанного продукта существенно улучшает качественные показатели, включая:

– снижение трудоёмкости: автоматизация процесса распознавания речи позволяет значительно сократить время, затрачиваемое на ручное транскрибирование аудио;

– сокращение затрат времени: продукт обеспечивает значительное уменьшение времени, необходимого для выполнения задач, что ведёт к экономии трудовых ресурсов и повышению общей эффективности.

Разработанный продукт обладает экономической и качественной эффективностью. Внедрение продукта снизит трудоёмкость и временные затраты на транскрибирование аудио. И будет экономически выгодным решением в выборе среди других аналогов на рынке.

7 Внедрение и установка информационной системы

Для установки программного обеспечения необходимо использовать операционную систему Windows 10 или Windows 11. Дополнительное программное обеспечение не требуется.

Программное обеспечение распространяется в виде портативной версии, что исключает необходимость установки. Все настройки программы и дополнительные файлы располагаются в каталоге «Materials», который включает следующие подкаталоги:

- AppSettings: настройки программного обеспечения;
- Dictionaries: словари для импорта и экспорта;
- Models: модели и их настройки;
- RecognizingAudio: аудиофайлы для тестирования;
- TrainingAudio: аудиофайлы для обучения.

Программа распространяется вместе с некоторым набором аудиофайлов, которые можно использовать для обучения модели. Эти файлы размещаются в двух основных каталогах:

TrainingAudio: каталог для аудиофайлов, участвующих в процессе обучения. Этот каталог разделяется на два подкаталога:

- Phonemes: содержит подкаталоги для каждой фонемы, названные заглавными буквами, олицетворяющими данную фонему (например, «Ль»);
- Words: содержит подкаталоги для каждого слова, названные заглавными буквами фонем, разделённых нижним подчёркиванием и олицетворяющими само слово (например, «Н_О_Ль»).

RecognizingAudio: каталог аудиофайлов, используемых в тестировании модели, имеет аналогичную структуру с каталогом TrainingAudio.

Помимо инструкции по установке системы, в Приложении Б представлено руководство пользователя.

					АТДП.09.02.07.20.404.ПЗ	Лист
						43
Изм	Лист	№ документа	Подпись			

Заключение

В ходе выполнения дипломного проекта была разработана система автоматизированного распознавания речи. Целью проекта было создание инструмента, позволяющего создавать модели распознавания речи, обучать на собственных аудио данных, формировать словари и использовать модели в транскрипции аудио.

Результат разработки подтверждает успешное достижение поставленной цели. Система демонстрирует способность к распознаванию речи, что делает её возможным в использовании для широкого спектра приложений, включая транскрибирование аудиозаписей, создание автоматизированных голосовых ассистентов и разработку систем распознавания команд голосового управления.

В ходе разработки также была подтверждена экономическая целесообразность проекта. Анализ тарифов конкурирующих компаний показал, что использование разработанной системы может привести к значительной экономии ресурсов при обработке аудиофайлов.

Дальнейшее развитие проекта может быть связано с расширением функциональности системы, включая поддержку иных алгоритмов извлечения качественных характеристик аудио и алгоритмов обучения моделей. Создание многоуровневого словаря для точного распознавания слов, различающихся в тексте и речи, например, «что» и «што».

Таким образом, разработанная система обладает потенциалом для широкого практического применения и дальнейшего развития.

					АТДП.09.02.07.20.404.ПЗ	Лист
						44
Изм	Лист	№ документа	Подпись			

Список использованных источников (литературы)

1. Бондаренко И.Ю. Распознавание речи: как сделать Speech-to-Text своими руками. [Электронный ресурс] / HighLoad, 2019. URL: <https://www.youtube.com/watch?v=eke2h9fGtu0&t> (дата обращения: 29.05.2024).
2. Краткое описание нотации BPMN. [Электронный ресурс] / Хабр, 2022. URL: <https://habr.com/ru/companies/auriga/articles/667084/> (дата обращения: 29.05.2024).
3. Краткий путеводитель по методологиям и нотациям описания и моделирования бизнес-процессов. Часть 2. [Электронный ресурс] / Инфостарт, 2021. URL: <https://infostart.ru/pm/1430187/> (дата обращения: 29.05.2024).
4. Максим Корневский - Введение в современное распознавание речи - DataStart.ru. [Электронный ресурс] / DataStart, 2020. URL: <https://www.youtube.com/watch?v=Y9Oav2tBxdw&t> (дата обращения: 29.05.2024).
5. Очень краткое введение в SysML. [Электронный ресурс] / Хабр, 2021. URL: <https://habr.com/ru/articles/542606/> (дата обращения: 29.05.2024).
6. Правила тарификации для SpeechKit. [Электронный ресурс] / Yandex Cloud, 2024. URL: https://yandex.cloud/ru/docs/speechkit/pricing?utm_referrer=https%3A%2F%2Fyandex.ru%2F (дата обращения: 29.05.2024).
7. Распознавание речи. Лекция 1. [Электронный ресурс] / Speech Technology Center, 2019. URL: <https://www.youtube.com/watch?v=BpTXT1nKNJ4&t> (дата обращения: 29.05.2024).
8. Распознавание речи. Лекция 2. [Электронный ресурс] / Speech Technology Center, 2019. URL:

<https://www.youtube.com/watch?v=PRaFNOmwUEg&t> (дата обращения: 29.05.2024).

9. Распознавание речи. Лекция 3. [Электронный ресурс] / Speech Technology Center, 2019. URL: <https://www.youtube.com/watch?v=XvUN1nIcBmQ&t> (дата обращения: 29.05.2024).

10. Что находится между идеей и кодом? Обзор 14 диаграмм UML. [Электронный ресурс] / Хабр, 2020. URL: <https://habr.com/ru/articles/508710/> (дата обращения: 29.05.2024).

11. Amazon Transcribe Pricing. . [Электронный ресурс] / aws, 2024. URL: <https://aws.amazon.com/transcribe/pricing> (дата обращения: 29.05.2024).

12. Azure AI Speech pricing. [Электронный ресурс] / Azure, 2024. URL: <https://azure.microsoft.com/en-us/pricing/details/cognitive-services/speech-services> (дата обращения: 29.05.2024).

13. Information technology - Object Management Group Unified Modeling Language (OMG UML), Infrastructure. [Электронный ресурс] / ISO/IEC 2024. URL: <https://www.omg.org/spec/UML/ISO/19505-1/PDF> (дата обращения: 29.05.2024).

14. Text-Independent Speaker Recognition System Using Feature-Level Fusion for Audio Databases of Various Sizes. [Электронный ресурс] / Springer Link, 2023. URL: <https://link.springer.com/article/10.1007/s42979-023-02056-w> (дата обращения: 29.05.2024).

15. Turn speech into text using Google AI. [Электронный ресурс] / Google Cloud, 2024. URL: <https://cloud.google.com/speech-to-text?hl=ru#pricing> (дата обращения: 29.05.2024).

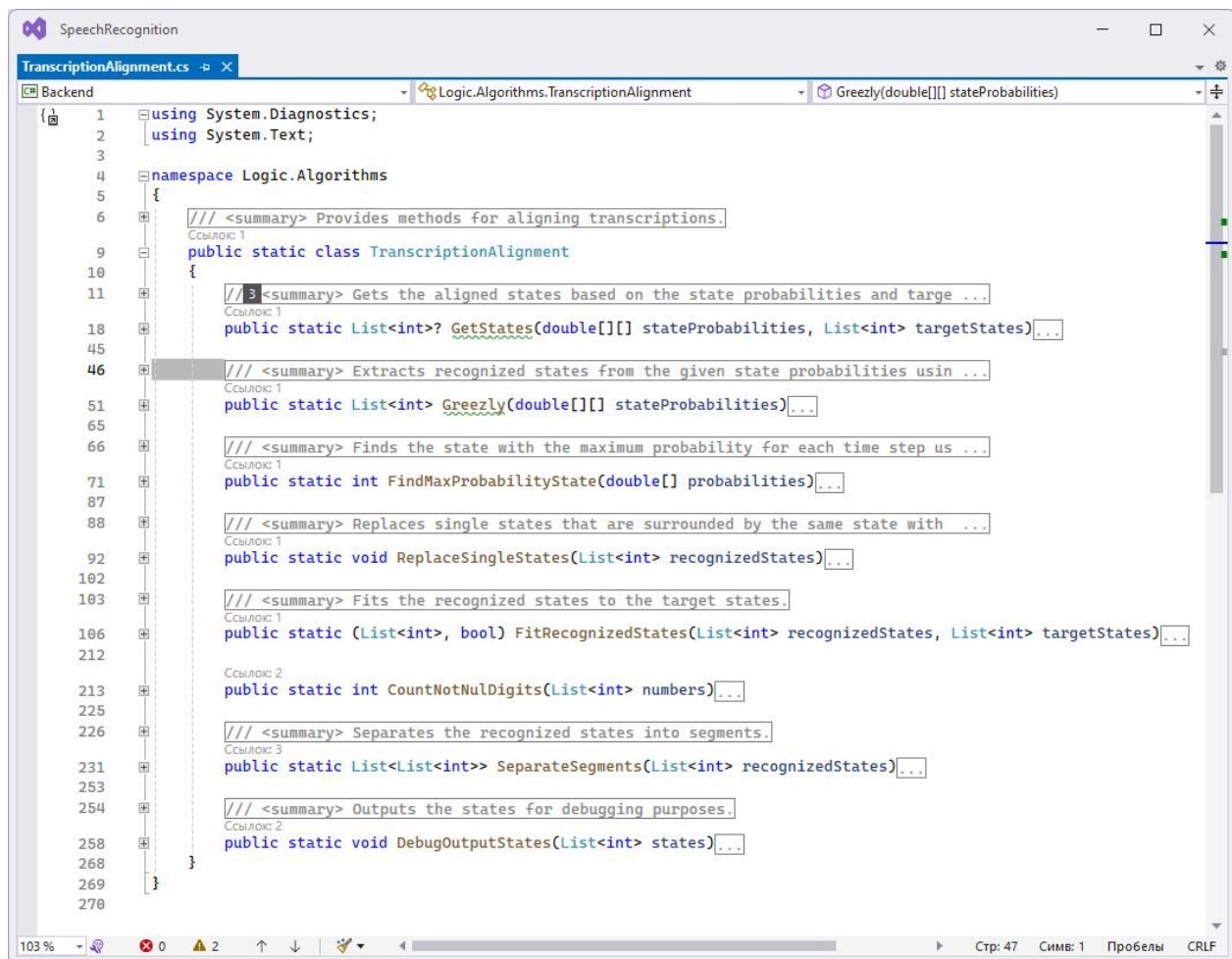
16. Voice2X. [Электронный ресурс] / ЦРТ, 2024. URL: <https://www.speechpro.ru/product/programmy-dlya-raspoznavaniya-rechi-v-tekst/voice2x> (дата обращения: 29.05.2024).

Приложение А

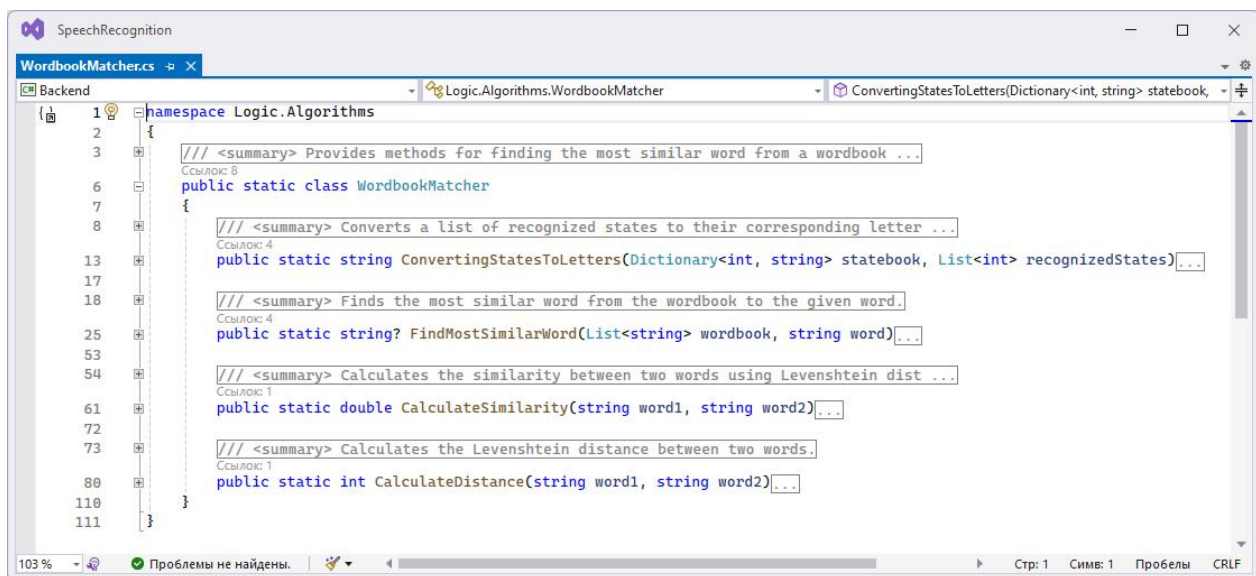
Код библиотеки

```
1 namespace Logic.Algorithms
2 {
3     /// <summary> Provides methods for processing state probabilities and extracting ...
4     Ссылка: 4
5     public static class StateProcessing
6     {
7         /// <summary> Processes the given state probabilities and returns a list of reco ...
8         Ссылка: 3
9         public static List<int> GetWordStates(double[][] stateProbabilities) ...
10
11         Ссылка: 1
12         public static List<List<int>> GetSentenceStates(double[][] stateProbabilities) ...
13
14         Ссылка: 1
15         private static List<int> TrimSilence(List<int> recognizedStates) ...
16
17         Ссылка: 1
18         private static List<List<int>> SeparateUnblankStates(List<int> recognizedStates) ...
19
20         /// <summary> Extracts recognized states from the given state probabilities usin ...
21         Ссылка: 2
22         private static List<int> Greezly(double[][] stateProbabilities) ...
23
24         /// <summary> Finds the state with the maximum probability for each time step us ...
25         Ссылка: 1
26         private static int FindMaxProbabilityState(double[] probabilities) ...
27
28         /// <summary> Replaces single states that are surrounded by the same state with ...
29         Ссылка: 5
30         public static void ReplaceSingleStates(List<int> recognizedStates) ...
31
32         /// <summary> Removes single states that are not preceded or followed by the sam ...
33         Ссылка: 2
34         public static void RemoveSingleStates(List<int> recognizedStates) ...
35
36         /// <summary> Removes blank states (state with index 0) from the list of recogni ...
37         Ссылка: 2
38         public static void RemoveBlankStates(List<int> recognizedStates) ...
39
40         /// <summary> Combines adjacent duplicate states into one state.
41         Ссылка: 2
42         public static void CombineStates(List<int> recognizedStates) ...
43     }
44 }
```

Продолжение приложения А

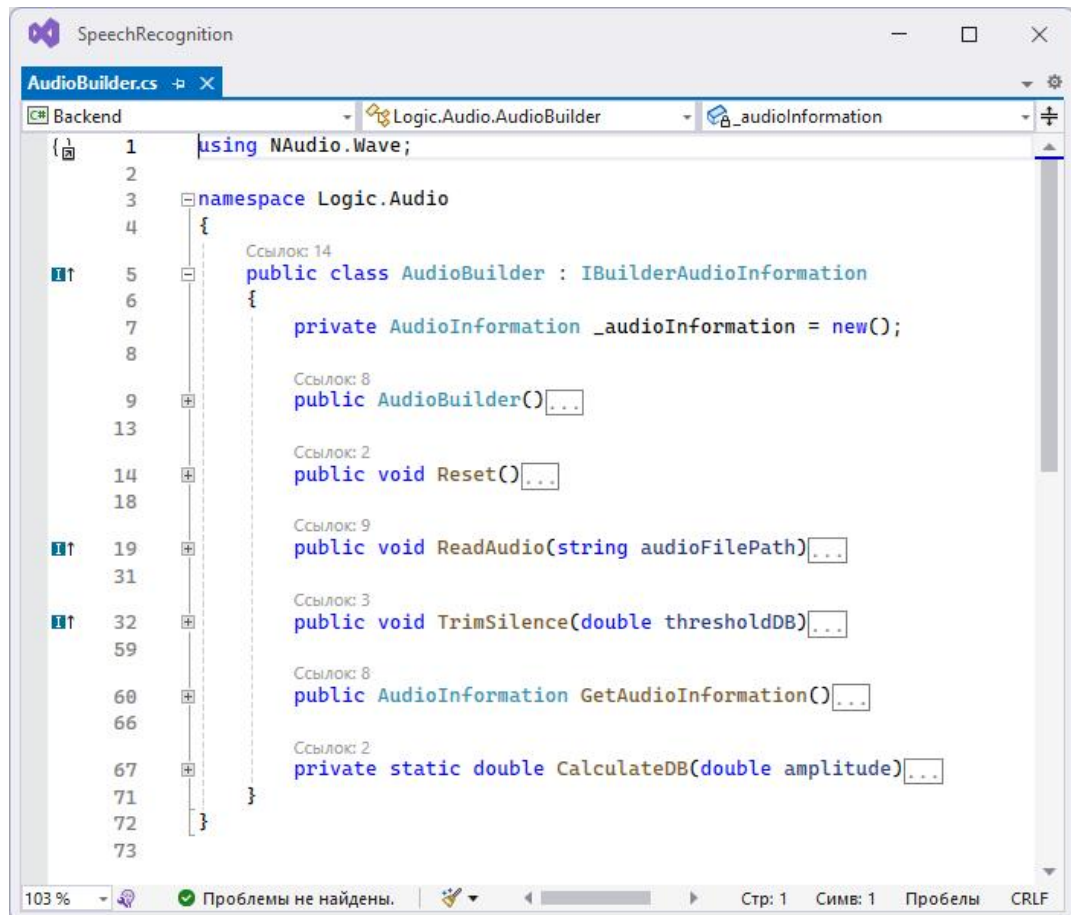


```
1 using System.Diagnostics;
2 using System.Text;
3
4 namespace Logic.Algorithms
5 {
6     /// <summary> Provides methods for aligning transcriptions.
7     Ссылка 1
8     public static class TranscriptionAlignment
9     {
10         /// <summary> Gets the aligned states based on the state probabilities and target ...
11         Ссылка 1
12         public static List<int>? GetStates(double[][] stateProbabilities, List<int> targetStates) ...
13
14         /// <summary> Extracts recognized states from the given state probabilities usin ...
15         Ссылка 1
16         public static List<int> Greezly(double[][] stateProbabilities) ...
17
18         /// <summary> Finds the state with the maximum probability for each time step us ...
19         Ссылка 1
20         public static int FindMaxProbabilityState(double[] probabilities) ...
21
22         /// <summary> Replaces single states that are surrounded by the same state with ...
23         Ссылка 1
24         public static void ReplaceSingleStates(List<int> recognizedStates) ...
25
26         /// <summary> Fits the recognized states to the target states.
27         Ссылка 1
28         public static (List<int>, bool) FitRecognizedStates(List<int> recognizedStates, List<int> targetStates) ...
29
30         Ссылка 2
31         public static int CountNotNulDigits(List<int> numbers) ...
32
33         /// <summary> Separates the recognized states into segments.
34         Ссылка 3
35         public static List<List<int>> SeparateSegments(List<int> recognizedStates) ...
36
37         /// <summary> Outputs the states for debugging purposes.
38         Ссылка 2
39         public static void DebugOutputStates(List<int> states) ...
40     }
41 }
```



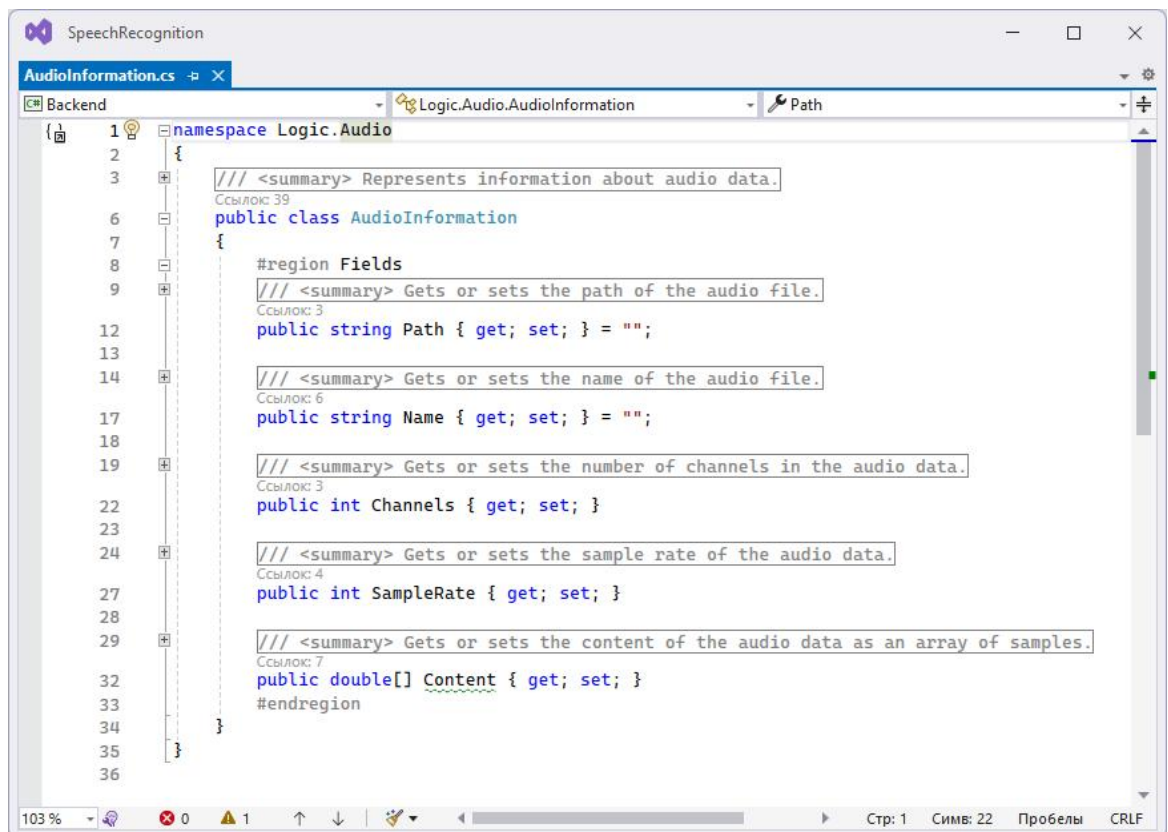
```
1 namespace Logic.Algorithms
2 {
3     /// <summary> Provides methods for finding the most similar word from a wordbook ...
4     Ссылка 8
5     public static class WordbookMatcher
6     {
7         /// <summary> Converts a list of recognized states to their corresponding letter ...
8         Ссылка 4
9         public static string ConvertingStatesToLetters(Dictionary<int, string> statebook, List<int> recognizedStates) ...
10
11         /// <summary> Finds the most similar word from the wordbook to the given word.
12         Ссылка 4
13         public static string? FindMostSimilarWord(List<string> wordbook, string word) ...
14
15         /// <summary> Calculates the similarity between two words using Levenshtein dist ...
16         Ссылка 1
17         public static double CalculateSimilarity(string word1, string word2) ...
18
19         /// <summary> Calculates the Levenshtein distance between two words.
20         Ссылка 1
21         public static int CalculateDistance(string word1, string word2) ...
22     }
23 }
```


Продолжение приложения А



```
1 using NAudio.Wave;
2
3 namespace Logic.Audio
4 {
5     Ссылка: 14
6     public class AudioBuilder : IBuilderAudioInformation
7     {
8         private AudioInformation _audioInformation = new();
9
10        Ссылка: 8
11        public AudioBuilder()...
12
13        Ссылка: 2
14        public void Reset()...
15
16        Ссылка: 9
17        public void ReadAudio(string audioFilePath)...
18
19        Ссылка: 3
20        public void TrimSilence(double thresholdDB)...
21
22        Ссылка: 8
23        public AudioInformation GetAudioInformation()...
24
25        Ссылка: 2
26        private static double CalculateDB(double amplitude)...
27    }
28 }
```

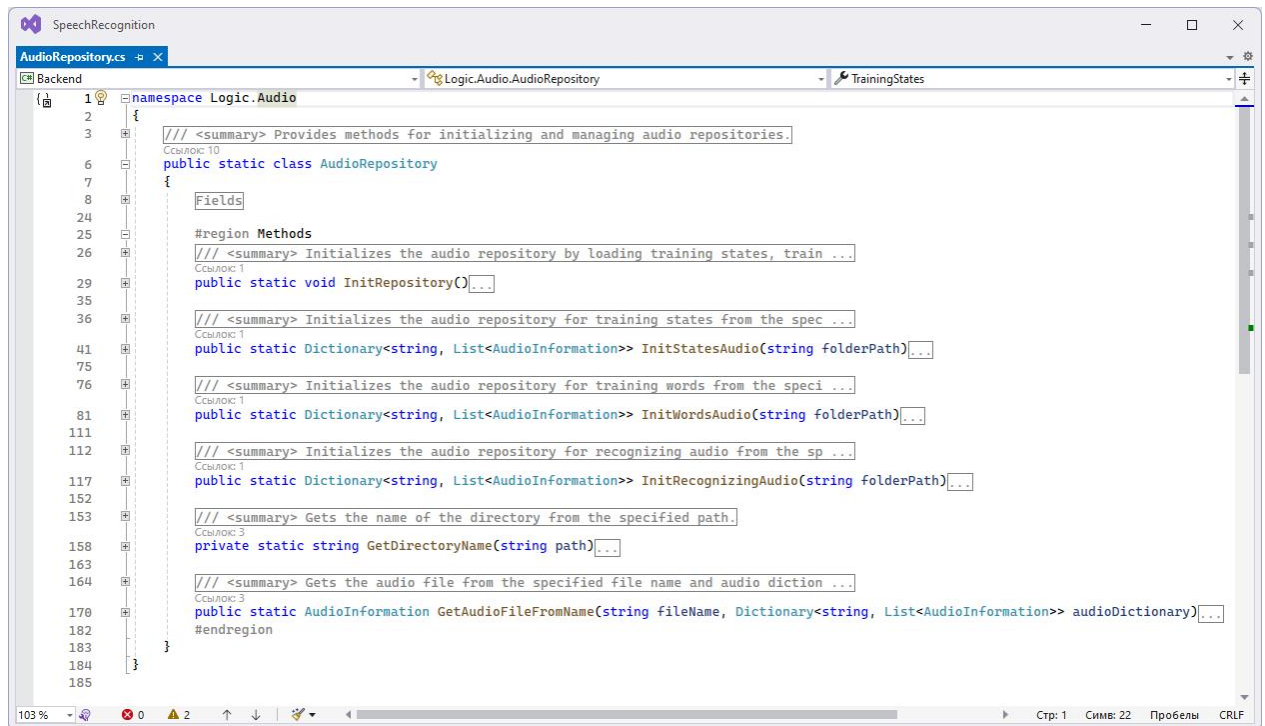
103 % Проблемы не найдены. Стр: 1 Симв: 1 Пробелы CRLF



```
1 namespace Logic.Audio
2 {
3     /// <summary> Represents information about audio data.
4     Ссылка: 39
5     public class AudioInformation
6     {
7         #region Fields
8         /// <summary> Gets or sets the path of the audio file.
9         Ссылка: 3
10        public string Path { get; set; } = "";
11
12        /// <summary> Gets or sets the name of the audio file.
13        Ссылка: 6
14        public string Name { get; set; } = "";
15
16        /// <summary> Gets or sets the number of channels in the audio data.
17        Ссылка: 3
18        public int Channels { get; set; }
19
20        /// <summary> Gets or sets the sample rate of the audio data.
21        Ссылка: 4
22        public int SampleRate { get; set; }
23
24        /// <summary> Gets or sets the content of the audio data as an array of samples.
25        Ссылка: 7
26        public double[] Content { get; set; }
27        #endregion
28    }
29 }
```

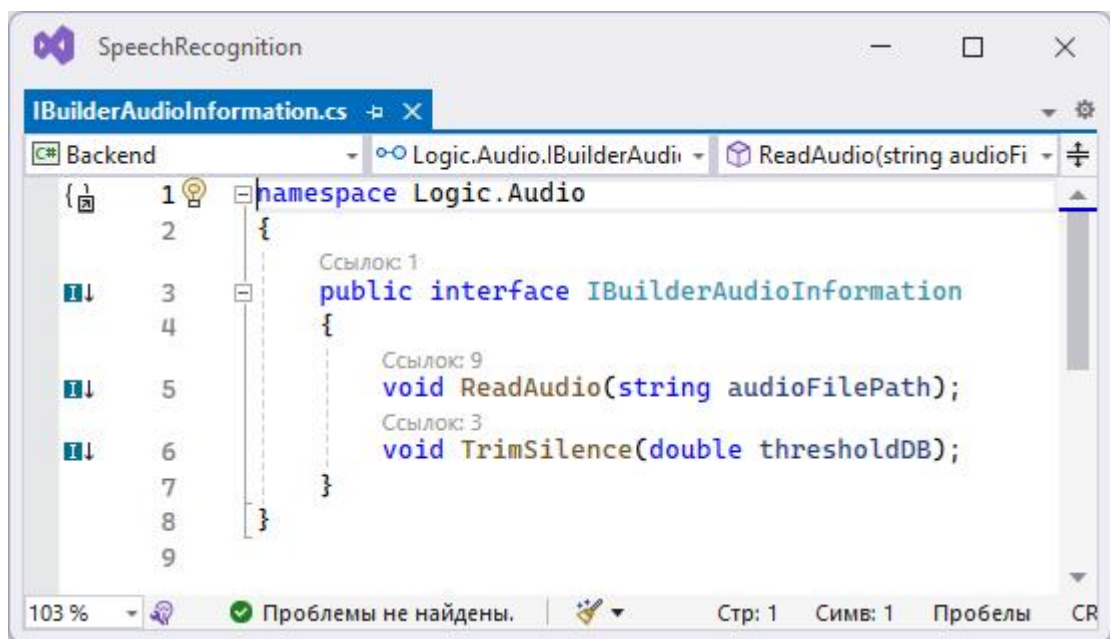
103 % 0 1 Стр: 1 Симв: 22 Пробелы CRLF

Продолжение приложения А



The screenshot shows the Visual Studio IDE with the file `AudioRepository.cs` open. The code is in the `namespace Logic.Audio`. It defines a `public static class AudioRepository` with several static methods for initializing and managing audio repositories. The methods include `InitRepository()`, `InitStatesAudio(string folderPath)`, `InitWordsAudio(string folderPath)`, `InitRecognizingAudio(string folderPath)`, `GetDirectoryName(string path)`, and `GetAudioFileFromName(string fileName, Dictionary<string, List<AudioInformation>> audioDictionary)`. The code is annotated with XML comments and region tags.

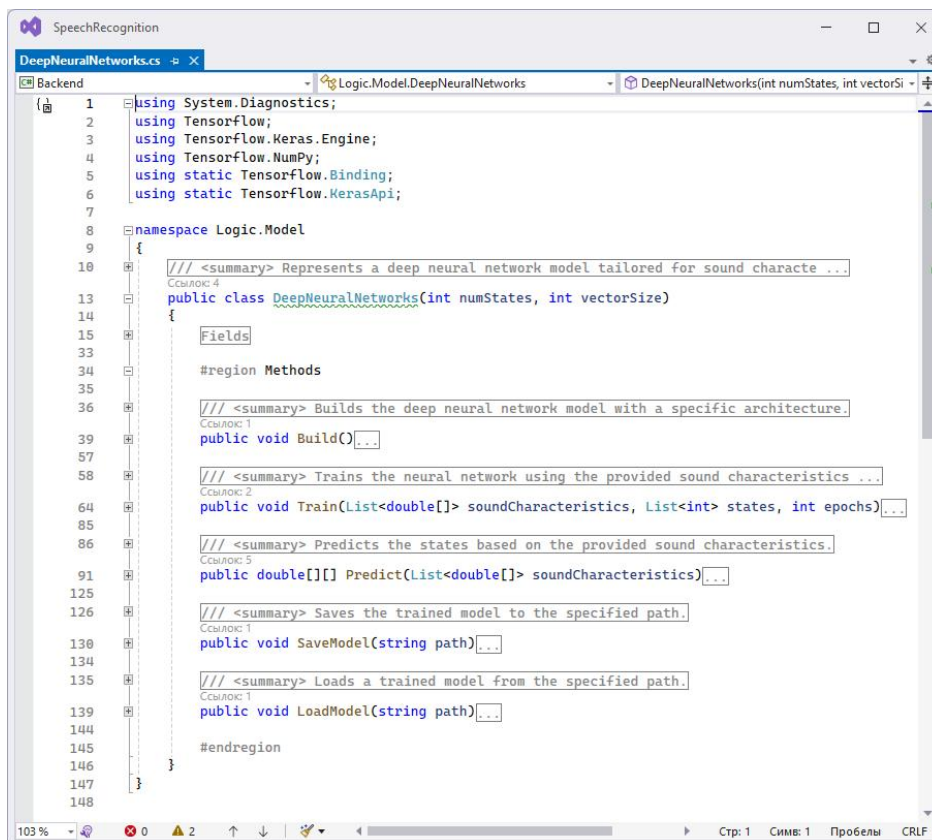
```
1 1 // <summary> Provides methods for initializing and managing audio repositories.
2 2
3 3
4 4
5 5
6 6 public static class AudioRepository
7 7 {
8 8     Fields
9 9
10 10
11 11 #region Methods
12 12 // <summary> Initializes the audio repository by loading training states, train ...
13 13 // Ссылка: 10
14 14 public static void InitRepository()...
15 15
16 16 // <summary> Initializes the audio repository for training states from the spec ...
17 17 // Ссылка: 1
18 18 public static Dictionary<string, List<AudioInformation>> InitStatesAudio(string folderPath)...
19 19
20 20 // <summary> Initializes the audio repository for training words from the speci ...
21 21 // Ссылка: 1
22 22 public static Dictionary<string, List<AudioInformation>> InitWordsAudio(string folderPath)...
23 23
24 24 // <summary> Initializes the audio repository for recognizing audio from the sp ...
25 25 // Ссылка: 1
26 26 public static Dictionary<string, List<AudioInformation>> InitRecognizingAudio(string folderPath)...
27 27
28 28 // <summary> Gets the name of the directory from the specified path.
29 29 private static string GetDirectoryName(string path)...
30 30
31 31 // <summary> Gets the audio file from the specified file name and audio dictio ...
32 32 // Ссылка: 3
33 33 public static AudioInformation GetAudioFileFromName(string fileName, Dictionary<string, List<AudioInformation>> audioDictionary)...
34 34 #endregion
35 35
36 36
37 37
38 38
39 39
40 40
41 41
42 42
43 43
44 44
45 45
46 46
47 47
48 48
49 49
50 50
51 51
52 52
53 53
54 54
55 55
56 56
57 57
58 58
59 59
60 60
61 61
62 62
63 63
64 64
65 65
66 66
67 67
68 68
69 69
70 70
71 71
72 72
73 73
74 74
75 75
76 76
77 77
78 78
79 79
80 80
81 81
82 82
83 83
84 84
85 85
```



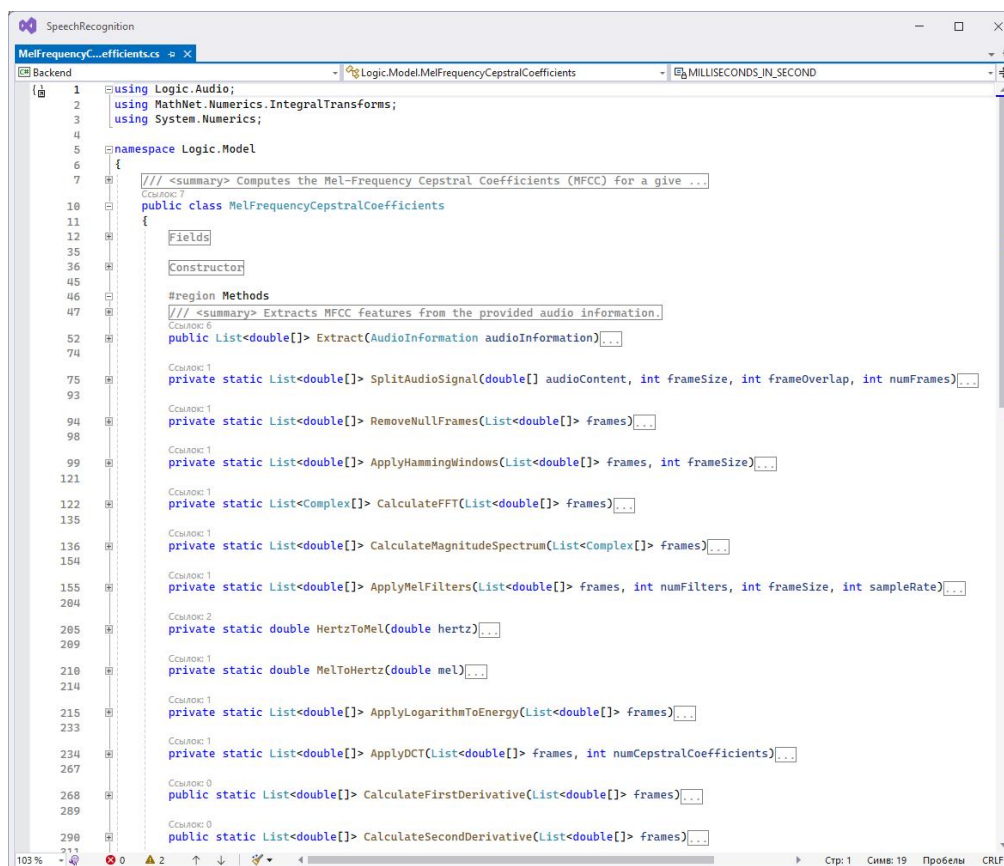
The screenshot shows the Visual Studio IDE with the file `IBuilderAudioInformation.cs` open. The code is in the `namespace Logic.Audio`. It defines a `public interface IBuilderAudioInformation` with two methods: `ReadAudio(string audioFilePath)` and `TrimSilence(double thresholdDB)`. The code is annotated with XML comments and region tags.

```
1 1 namespace Logic.Audio
2 2 {
3 3     // Ссылка: 1
4 4     public interface IBuilderAudioInformation
5 5     {
6 6         // Ссылка: 9
7 7         void ReadAudio(string audioFilePath);
8 8         // Ссылка: 3
9 9         void TrimSilence(double thresholdDB);
10 10     }
11 11 }
12 12
```

Продолжение приложения А

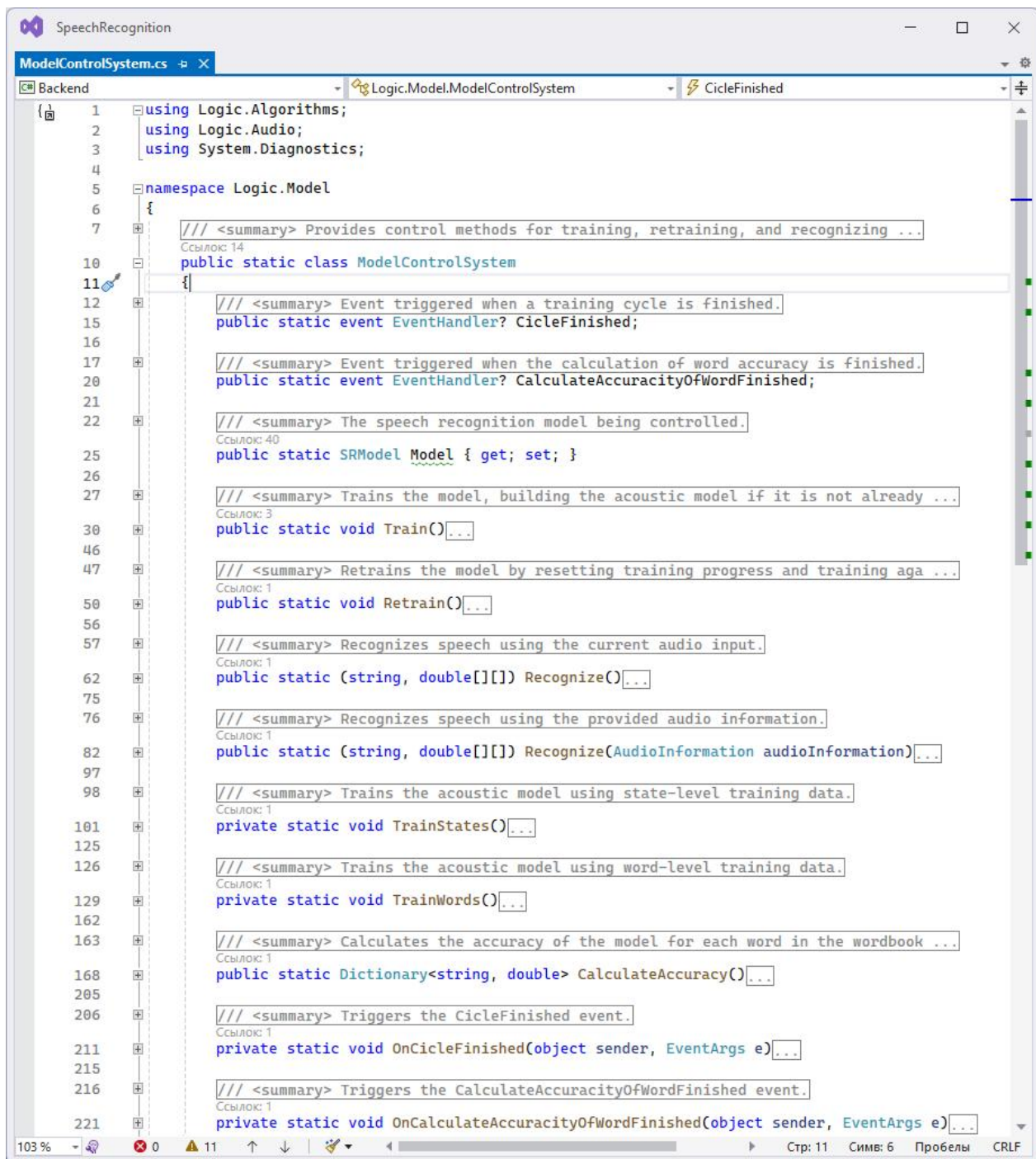


```
1 using System.Diagnostics;
2 using TensorFlow;
3 using TensorFlow.Keras.Engine;
4 using TensorFlow.NumPy;
5 using static TensorFlow.Binding;
6 using static TensorFlow.KerasApi;
7
8 namespace Logic.Model
9 {
10     /// <summary> Represents a deep neural network model tailored for sound characte ...
11     Cсылка: 4
12     public class DeepNeuralNetworks(int numStates, int vectorSize)
13     {
14         Fields
15
16         #region Methods
17
18         /// <summary> Builds the deep neural network model with a specific architecture.
19         Cсылка: 1
20         public void Build()...
21
22         /// <summary> Trains the neural network using the provided sound characteristics ...
23         Cсылка: 2
24         public void Train(List<double[]> soundCharacteristics, List<int> states, int epochs)...
25
26         /// <summary> Predicts the states based on the provided sound characteristics.
27         Cсылка: 5
28         public double[][] Predict(List<double[]> soundCharacteristics)...
29
30         /// <summary> Saves the trained model to the specified path.
31         Cсылка: 1
32         public void SaveModel(string path)...
33
34         /// <summary> Loads a trained model from the specified path.
35         Cсылка: 1
36         public void LoadModel(string path)...
37
38         #endregion
39     }
40 }
```



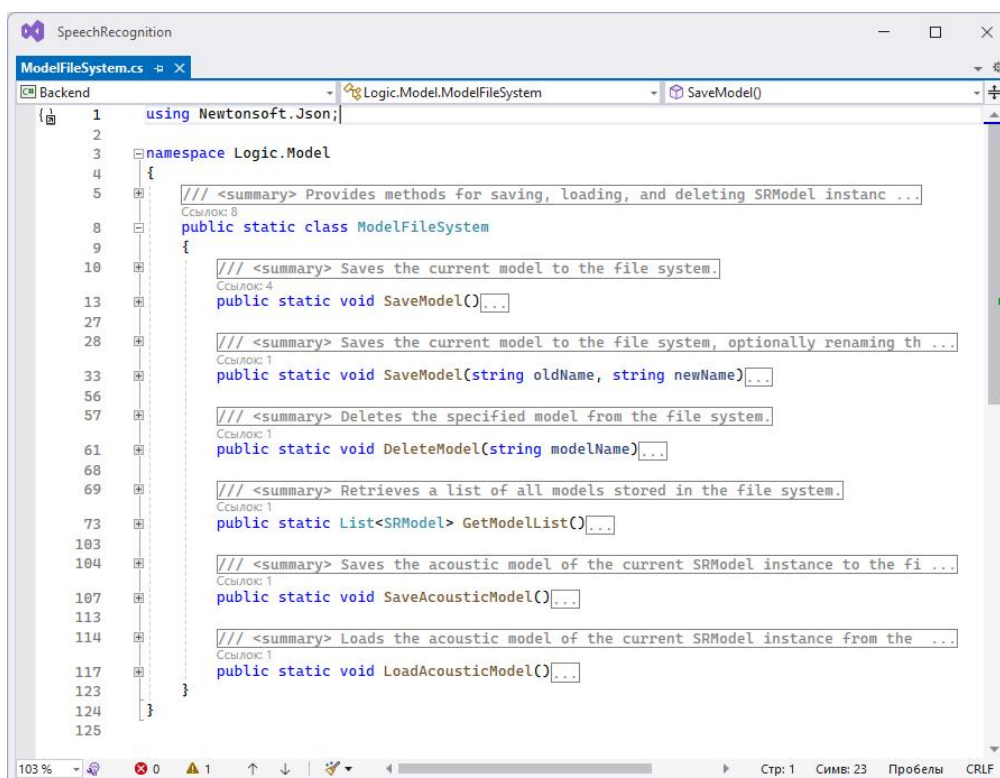
```
1 using Logic.Audio;
2 using MathNet.Numerics.IntegralTransforms;
3 using System.Numerics;
4
5 namespace Logic.Model
6 {
7     /// <summary> Computes the Mel-Frequency Cepstral Coefficients (MFCC) for a give ...
8     Cсылка: 7
9     public class MelFrequencyCepstralCoefficients
10     {
11         Fields
12
13         Constructor
14
15         #region Methods
16
17         /// <summary> Extracts MFCC features from the provided audio information.
18         Cсылка: 6
19         public List<double[]> Extract(AudioInformation audioInformation)...
20
21         private static List<double[]> SplitAudioSignal(double[] audioContent, int frameSize, int frameOverlap, int numFrames)...
22
23         private static List<double[]> RemoveNullFrames(List<double[]> frames)...
24
25         private static List<double[]> ApplyHammingWindows(List<double[]> frames, int frameSize)...
26
27         private static List<Complex[]> CalculateFFT(List<double[]> frames)...
28
29         private static List<double[]> CalculateMagnitudeSpectrum(List<Complex[]> frames)...
30
31         private static List<double[]> ApplyMelFilters(List<double[]> frames, int numFilters, int frameSize, int sampleRate)...
32
33         private static double HertzToMel(double hertz)...
34
35         private static double MelToHertz(double mel)...
36
37         private static List<double[]> ApplyLogarithmToEnergy(List<double[]> frames)...
38
39         private static List<double[]> ApplyDCT(List<double[]> frames, int numCepstralCoefficients)...
40
41         public static List<double[]> CalculateFirstDerivative(List<double[]> frames)...
42
43         public static List<double[]> CalculateSecondDerivative(List<double[]> frames)...
```

Продолжение приложения А

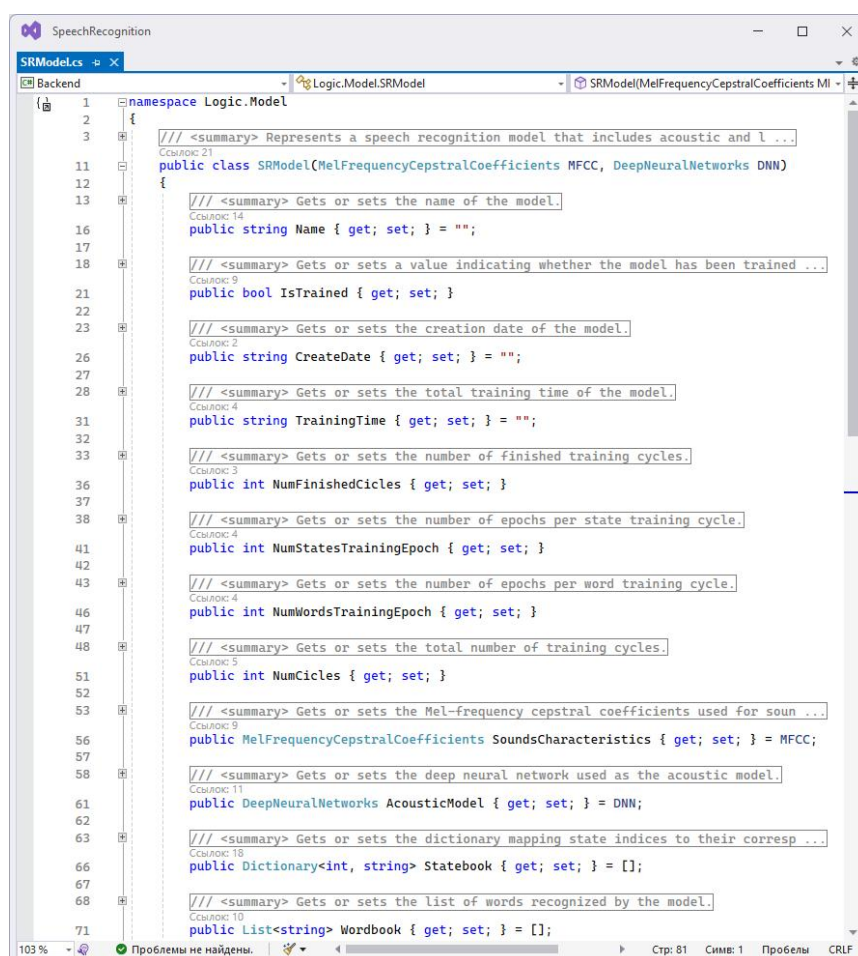


```
1  using Logic.Algorithms;
2  using Logic.Audio;
3  using System.Diagnostics;
4
5  namespace Logic.Model
6  {
7      /// <summary> Provides control methods for training, retraining, and recognizing ...
8      Ссылка: 14
9      public static class ModelControlSystem
10     {
11
12         /// <summary> Event triggered when a training cycle is finished.
13         public static event EventHandler? CicleFinished;
14
15         /// <summary> Event triggered when the calculation of word accuracy is finished.
16         public static event EventHandler? CalculateAccuracyOfWordFinished;
17
18         /// <summary> The speech recognition model being controlled.
19         Ссылка: 40
20         public static SRModel Model { get; set; }
21
22         /// <summary> Trains the model, building the acoustic model if it is not already ...
23         Ссылка: 3
24         public static void Train()...
25
26         /// <summary> Retrains the model by resetting training progress and training aga ...
27         Ссылка: 1
28         public static void Retrain()...
29
30         /// <summary> Recognizes speech using the current audio input.
31         Ссылка: 1
32         public static (string, double[][]) Recognize()...
33
34         /// <summary> Recognizes speech using the provided audio information.
35         Ссылка: 1
36         public static (string, double[][]) Recognize(AudioInformation audioInformation)...
37
38         /// <summary> Trains the acoustic model using state-level training data.
39         Ссылка: 1
40         private static void TrainStates()...
41
42         /// <summary> Trains the acoustic model using word-level training data.
43         Ссылка: 1
44         private static void TrainWords()...
45
46         /// <summary> Calculates the accuracy of the model for each word in the wordbook ...
47         Ссылка: 1
48         public static Dictionary<string, double> CalculateAccuracy()...
49
50         /// <summary> Triggers the CicleFinished event.
51         Ссылка: 1
52         private static void OnCicleFinished(object sender, EventArgs e)...
53
54         /// <summary> Triggers the CalculateAccuracyOfWordFinished event.
55         Ссылка: 1
56         private static void OnCalculateAccuracyOfWordFinished(object sender, EventArgs e)...
```


Окончание приложения А



```
1 using Newtonsoft.Json;
2
3 namespace Logic.Model
4 {
5     /// <summary> Provides methods for saving, loading, and deleting SRModel instanc ...
6     Ссылка: 8
7     public static class ModelFileSystem
8     {
9         /// <summary> Saves the current model to the file system.
10        Ссылка: 4
11        public static void SaveModel()...
12
13        /// <summary> Saves the current model to the file system, optionally renaming th ...
14        Ссылка: 1
15        public static void SaveModel(string oldName, string newName) ...
16
17        /// <summary> Deletes the specified model from the file system.
18        Ссылка: 1
19        public static void DeleteModel(string modelName) ...
20
21        /// <summary> Retrieves a list of all models stored in the file system.
22        Ссылка: 1
23        public static List<SRModel> GetModelList() ...
24
25        /// <summary> Saves the acoustic model of the current SRModel instance to the fi ...
26        Ссылка: 1
27        public static void SaveAcousticModel() ...
28
29        /// <summary> Loads the acoustic model of the current SRModel instance from the ...
30        Ссылка: 1
31        public static void LoadAcousticModel() ...
32    }
33 }
34
35 103 % 0 1 100 % Стр: 1 Симв: 23 Пробелы CRLF
```



```
1 namespace Logic.Model
2 {
3     /// <summary> Represents a speech recognition model that includes acoustic and l ...
4     Ссылка: 21
5     public class SRModel(MelFrequencyCepstralCoefficients MFCC, DeepNeuralNetworks DNN)
6     {
7         /// <summary> Gets or sets the name of the model.
8         Ссылка: 14
9         public string Name { get; set; } = "";
10
11        /// <summary> Gets or sets a value indicating whether the model has been trained ...
12        Ссылка: 9
13        public bool IsTrained { get; set; }
14
15        /// <summary> Gets or sets the creation date of the model.
16        Ссылка: 2
17        public string CreateDate { get; set; } = "";
18
19        /// <summary> Gets or sets the total training time of the model.
20        Ссылка: 4
21        public string TrainingTime { get; set; } = "";
22
23        /// <summary> Gets or sets the number of finished training cycles.
24        Ссылка: 3
25        public int NumFinishedCicles { get; set; }
26
27        /// <summary> Gets or sets the number of epochs per state training cycle.
28        Ссылка: 4
29        public int NumStatesTrainingEpoch { get; set; }
30
31        /// <summary> Gets or sets the number of epochs per word training cycle.
32        Ссылка: 4
33        public int NumWordsTrainingEpoch { get; set; }
34
35        /// <summary> Gets or sets the total number of training cycles.
36        Ссылка: 5
37        public int NumCicles { get; set; }
38
39        /// <summary> Gets or sets the Mel-frequency cepstral coefficients used for soun ...
40        Ссылка: 9
41        public MelFrequencyCepstralCoefficients SoundsCharacteristics { get; set; } = MFCC;
42
43        /// <summary> Gets or sets the deep neural network used as the acoustic model.
44        Ссылка: 11
45        public DeepNeuralNetworks AcousticModel { get; set; } = DNN;
46
47        /// <summary> Gets or sets the dictionary mapping state indices to their corres ...
48        Ссылка: 18
49        public Dictionary<int, string> Statebook { get; set; } = [];
50
51        /// <summary> Gets or sets the list of words recognized by the model.
52        Ссылка: 10
53        public List<string> Wordbook { get; set; } = [];
54    }
55 }
56
57 103 % Проблемы не найдены. Стр: 81 Симв: 1 Пробелы CRLF
```

Приложение Б

Руководство пользователя

Для запуска программы дважды щёлкните на значок программы. После запуска откроется окно программы, состоящее из трёх компонентов.

Первый компонент — панель инструментов, находящаяся в верхней части программы и содержащая кнопки для открытия окна и страниц программы. На рисунке Б.1 изображена панель инструментов.

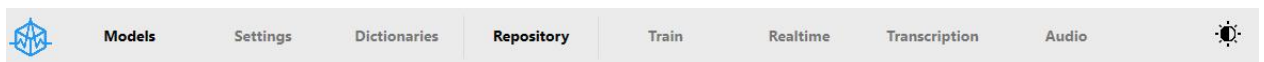


Рисунок Б.1 — Панель инструментов

Помимо кнопок перехода на страницы и открытия окон, панель инструментов содержит кнопку, которая позволяет изменять тему интерфейса программы с тёмной на светлую и наоборот. На рисунке Б.2 изображена кнопка смены темы программы.



Рисунок Б.2 — Кнопка смены темы программы

Второй компонент — контейнер содержимого страницы, находящийся в центральной части программы. Отображает содержимое страницы на которой находится пользователь. На рисунке 3 изображён контейнер содержимого стартовой страницы.

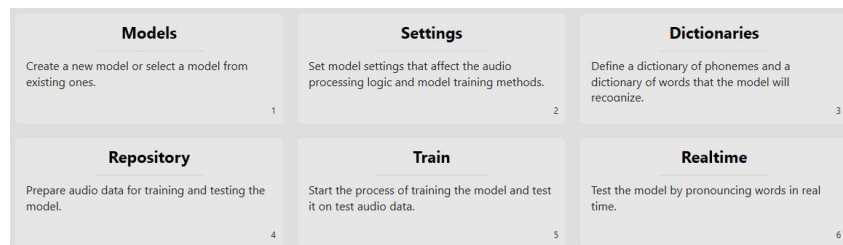


Рисунок Б.3 — Контейнер содержимого стартовой страницы

На стартовой странице отображается основная информация о функциональности и назначении страниц и окон.

Продолжение приложения Б

Третий компонент — строка состояния, находящаяся в нижней части программы. Отображает выбранную модель, её состояние обучения и бар прогресса при выполнении некоторых процессов. На рисунке Б.4 изображена строка состояния.

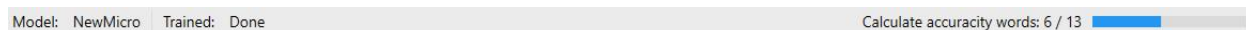


Рисунок Б.4 — Строка состояния

На выбранном примере строка состояния отображает название текущей модели «NewMicro», состояние обучения модели «Done» и бар прогресса расчёта точности распознавания слов.

Программное обеспечение позволяет создавать модели, обучать их, тестировать на точность распознавания и использовать в транскрибировании аудио. Все этапы работы с моделями происходят последовательно и разбиты на отдельные страницы и окна. Всю работу с моделями можно разбить на шаги.

Первый шаг — создание новой модели. Для создания модели нажмите кнопку «Models». Откроется окно, где можно нажать «Create new model» для создания новой модели или выбрать существующую, если такие модели имеются. Также можно указать путь к директории для хранения моделей, введя его в текстовое поле или выбрав через диалоговое окно. На рисунке Б.5 изображено окно списка моделей.

Продолжение приложения Б

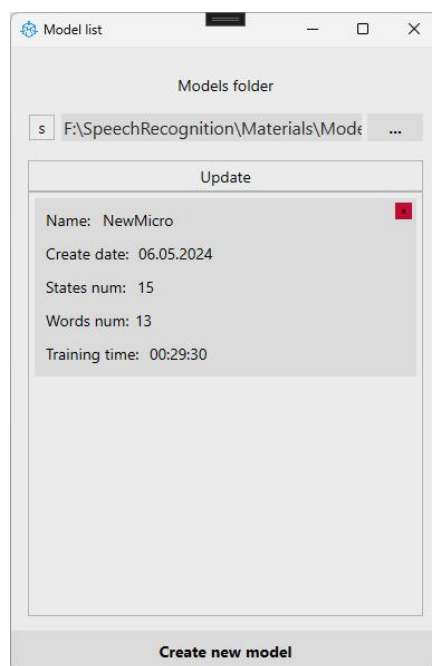


Рисунок Б.5 — Окно списка моделей

Также можно удалить существующую модель в списке нажав на крестик в красном квадрате, находящиеся в верхнем правом углу модели. При попытке удалить, вылезет окно с подтверждением действия. На рисунке Б.6 изображено окно подтверждения удаления модели.

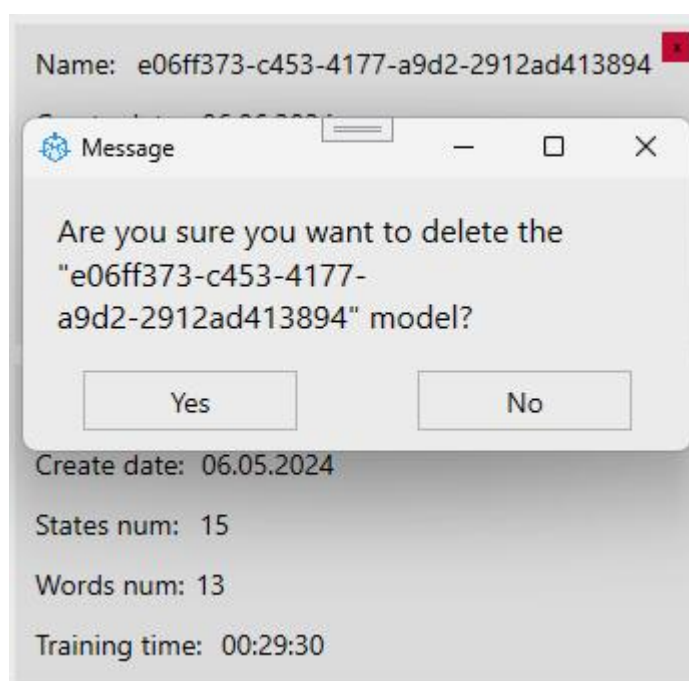


Рисунок Б.6 — Окно подтверждения удаления модели

Продолжение приложения Б

Второй шаг — настройка модели. Нажмите «Settings» на панели инструментов, чтобы открыть окно настроек модели. Здесь можно задать имя модели и настроить параметры MFCC и DNN. Для сохранения настроек нажмите «Apply». На рисунке Б.7 изображено окно настройки модели.

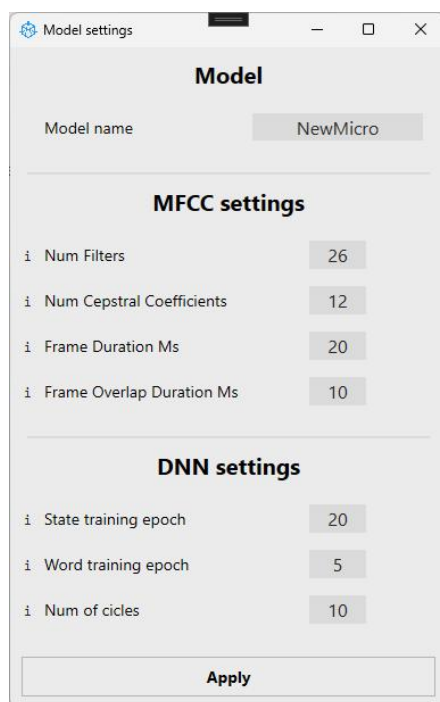


Рисунок Б.7 — Окно настройки модели

У каждого параметра в окне настройки модели есть подсказка, при наведении на которую появится дополнительная информация, описывающее назначение параметра. На рисунке Б.8 изображена подсказка параметра модели.

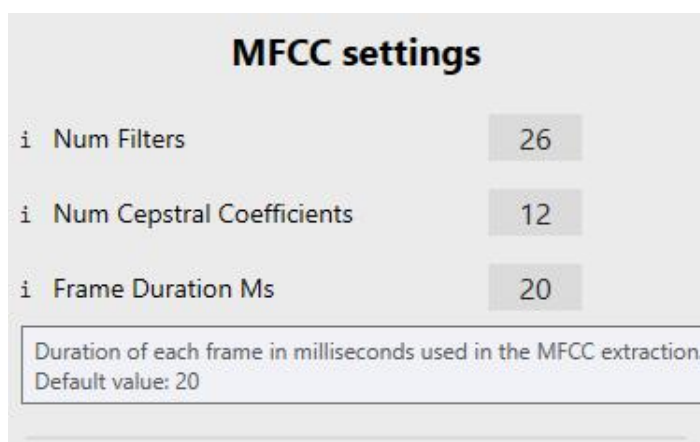


Рисунок Б.8 — Подсказка параметра модели

Продолжение приложения Б

Третий шаг — создание словарей. Нажмите «Dictionaries» на панели инструментов, чтобы открыть окно с двумя полями ввода: одно для словаря фонем (минимальные единицы языка), другое — для словаря слов. Фонемы и слова вводятся построчно. Также можно импортировать словари, нажав кнопку «Import» над соответствующим полем. Для сохранения словарей нажмите «Apply». На рисунке Б.9 изображено окно словарей модели.

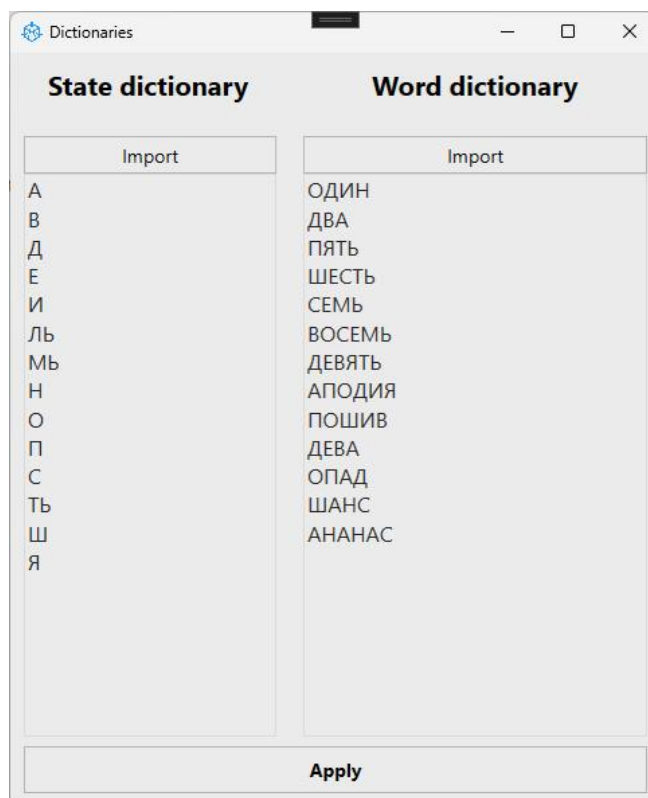


Рисунок Б.9 — Окно словарей модели

Четвёртый шаг — инициализация аудиофайлов. Нажмите «Repositories» на панели инструментов. Откроется страница, где можно указать пути к трём каталогам для аудио репозитория: фонемы, слова и тестовые данные. Введите пути в соответствующие текстовые поля или выберите их через диалоговое окно. Для инициализации аудиофайлов нажмите «Init Repositories». На рисунке Б.10 изображена страница инициализации аудио файлов.

Продолжение приложения Б

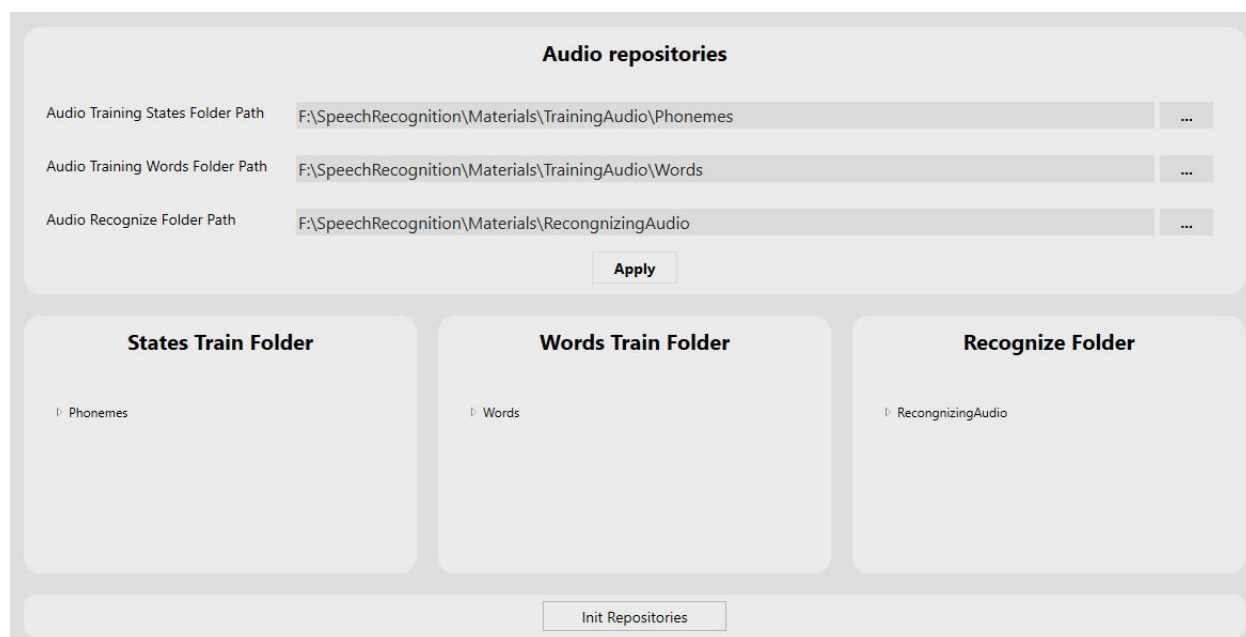


Рисунок Б.10 — Страница инициализации аудио файлов

Пятый шаг — обучение модели. Нажмите «Train» на панели инструментов, чтобы открыть страницу для обучения и тестирования модели. На рисунке Б.11 изображена страница обучения модели.

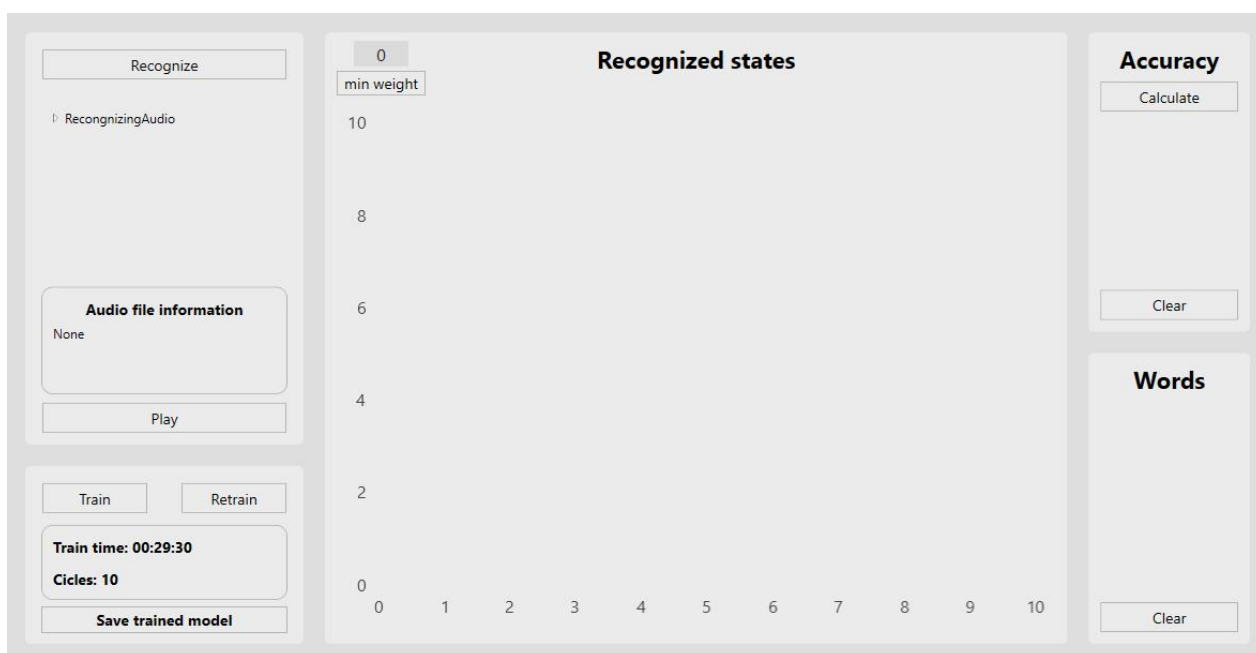


Рисунок Б.11 — Страница обучения модели

Окончание приложения Б

Нажмите «Train» для запуска процесса обучения, прогресс будет отображаться на панели состояния. Модель можно переобучить, нажав «Retrain», и сохранить, нажав «Save trained model». Для распознавания аудио выберите необходимый файл в дереве инициализированных данных и нажмите «Recognize». Отобразится матрица вероятностей и распознанное слово. Для расчёта точности распознавания нажмите «Calculate».

Шестой шаг — использование модели в транскрипции. Нажмите «Transcription» на панели инструментов, чтобы открыть страницу транскрибирования аудио. Нажмите на «Transcribe» чтобы выбрать аудио файл из файловой системы и сформировать транскрипцию для него. После формирования транскрипции, она отобразится в текстовом поле ниже. Нажмите кнопку «Save» чтобы сохранить транскрипцию в текстовом файле. На рисунке Б.12 изображено окно транскрибирования аудио.

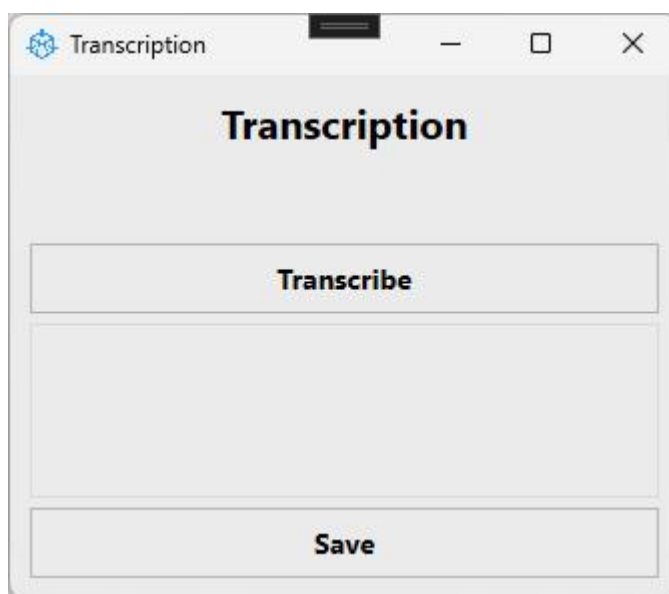


Рисунок Б.12 — Окно транскрибирования аудио

Обученную модель можно интегрировать в другие информационные системы, использующие библиотеку разработанного программного обеспечения.